

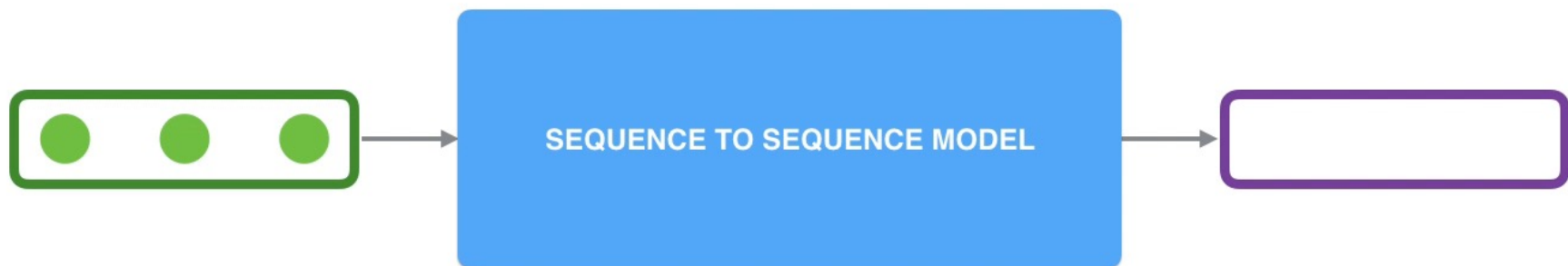
Towards BERT



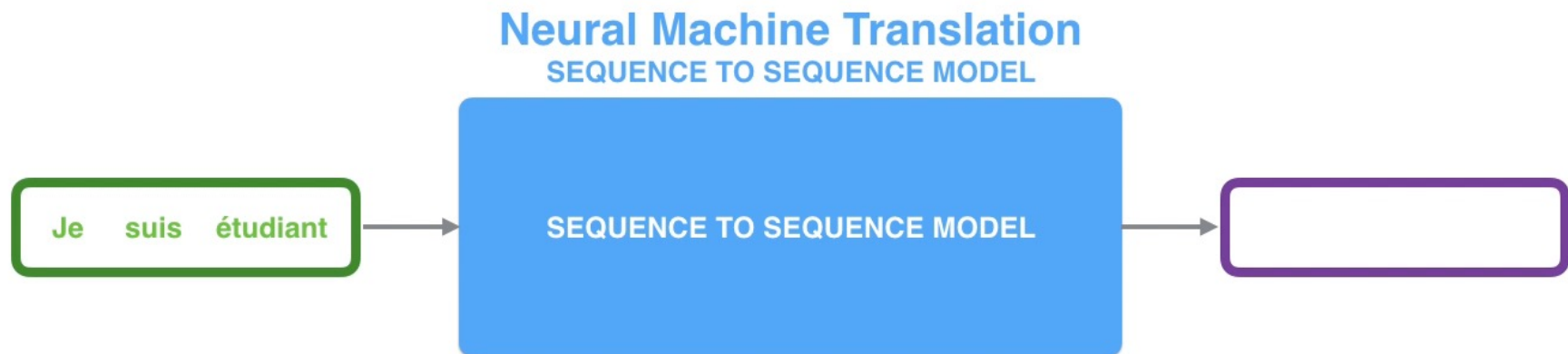
Attention Please !

Sequence-to-Sequence model

A sequence-to-sequence model is a model that takes a sequence of items (words, letters, features of an images...etc) and outputs another sequence of items.



Neural Machine Translation



Encoder-Decoder

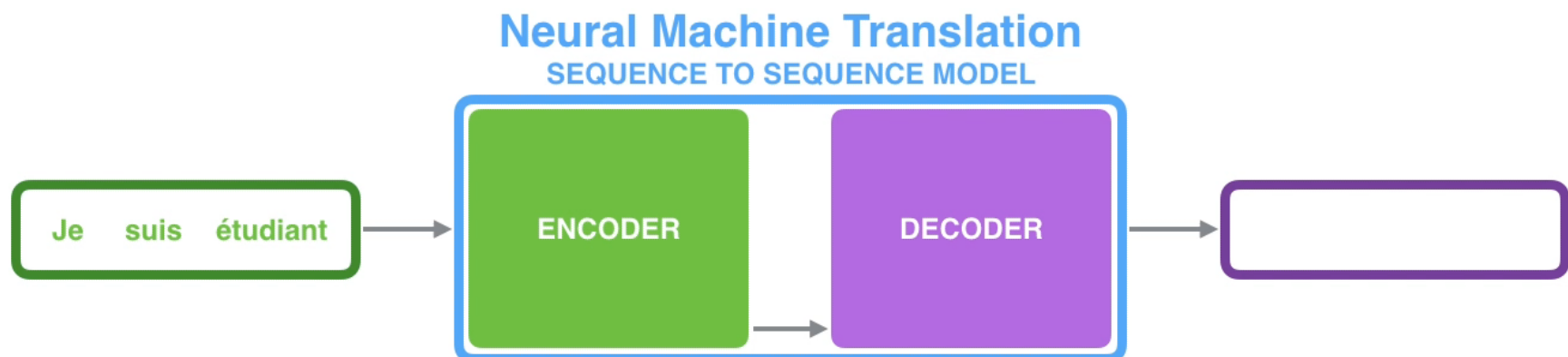
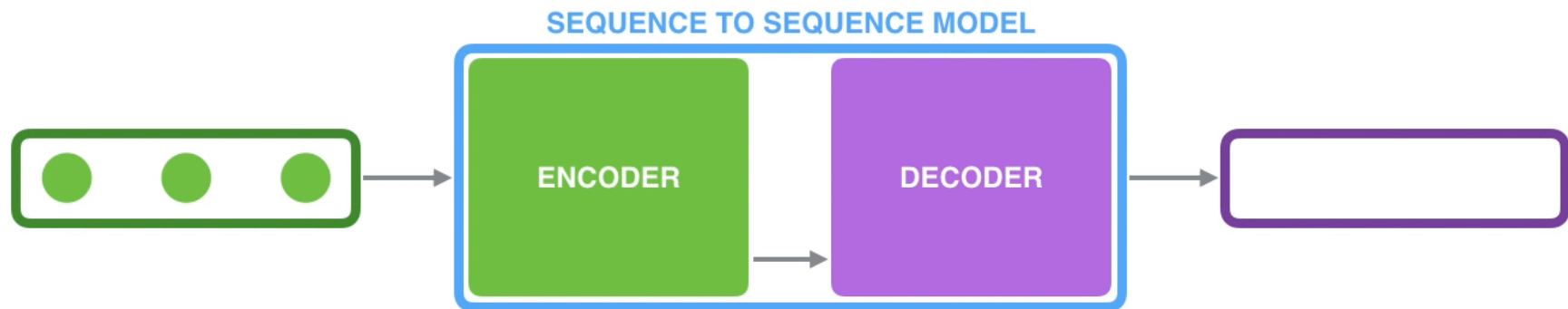
A sequence-to-sequence model is a model that contains an **Encoder** and **Decoder**.

The **encoder** processes each item in the input sequence, it compiles the information it captures into a vector (called the **context**).

After processing the entire input sequence, the **encoder** send the **context** over to the **decoder**, which begins producing the output sequence item by item.

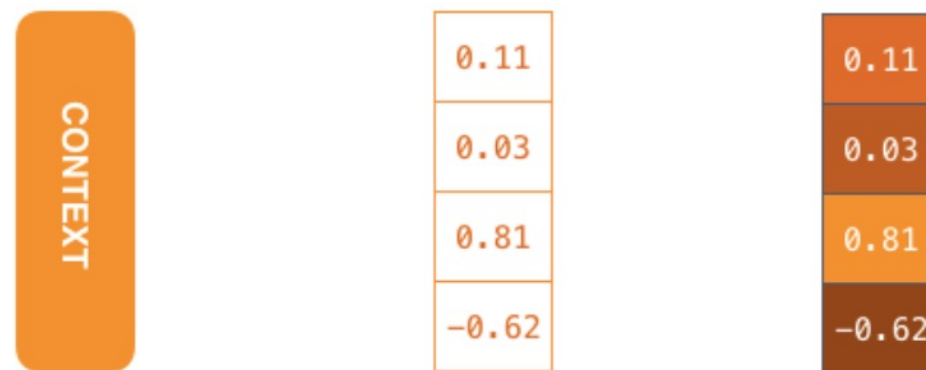


Encoder-Decoder



Context

The **context** is a vector (an array of numbers, basically) in the case of machine translation. The **encoder** and **decoder** tend to both be recurrent neural networks.



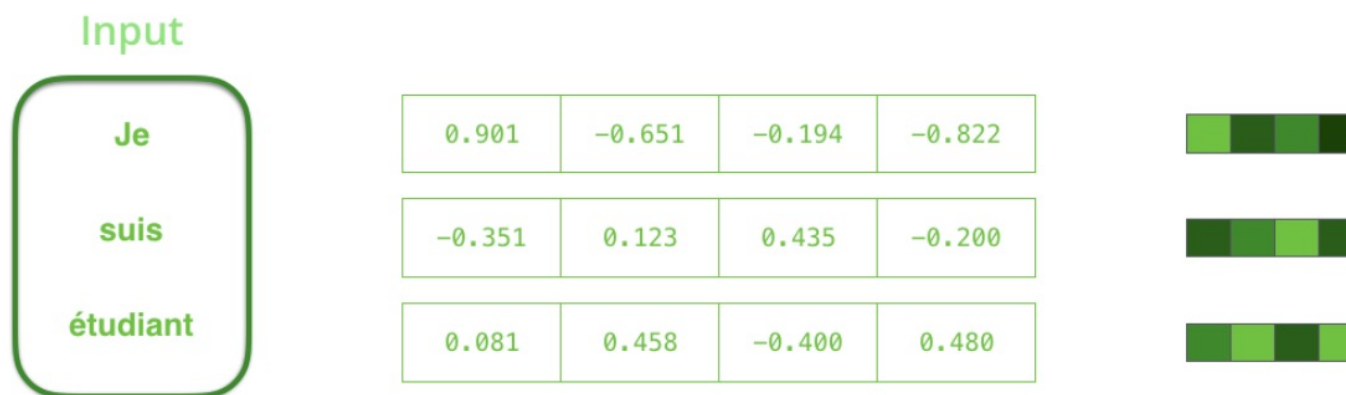
You can set the size of the **context** vector when you set up your model. It is basically the number of hidden units in the **encoder** RNN. In real world

applications the **context** vector would be of a size like 256, 512, or 1024.



Word Embeddings

A RNN takes two inputs at each time step: an **input** (in the case of the encoder, one word from the input sentence), and a **hidden state**. The word, however, needs to be represented by a vector. To transform a word into a vector, use “[word embedding](#)” algorithms



Recurrent Neural Network

Recurrent Neural Network

Time step #1:

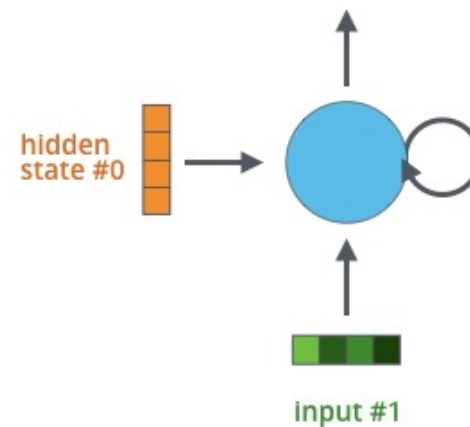
An RNN takes two input vectors:



hidden
state #0



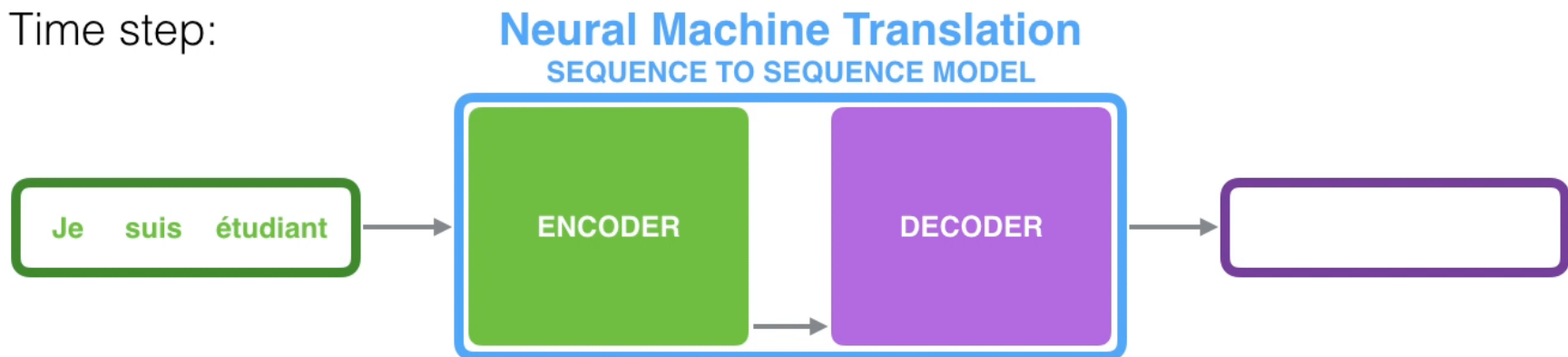
input vector #1



Time steps

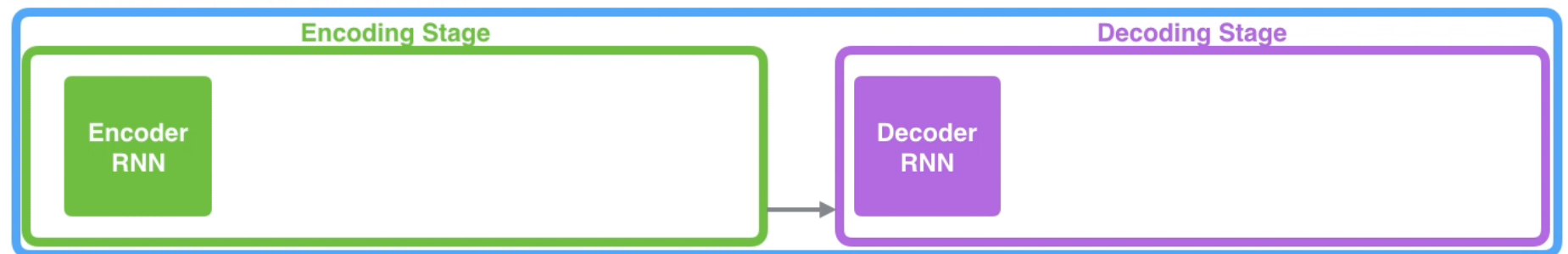
RNN processes its inputs and generating an output for that time step. Since the **encoder** and **decoder** are both RNNs, each time step one of the RNNs does some processing, it updates its **hidden state** based on its inputs and previous inputs it has seen.

Time step:



Unrolled view of RNN

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



Je

suis

étudiant



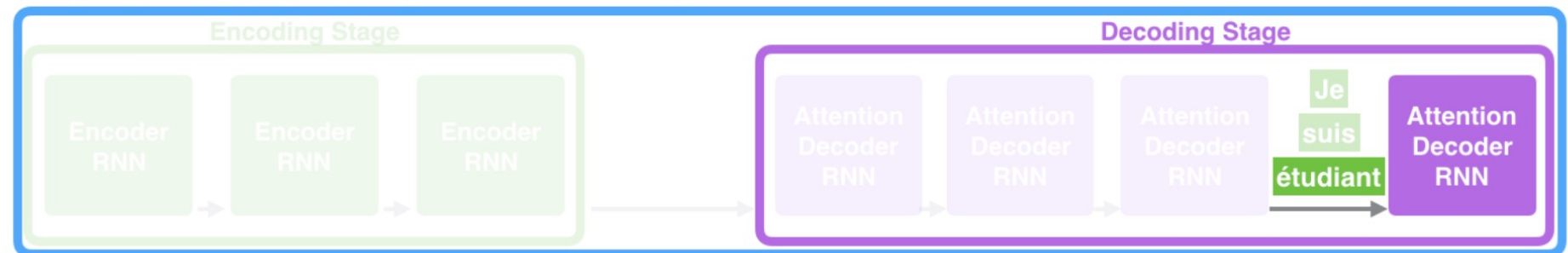
Attention please!

The **context** vector turned out to be a bottleneck for these types of models. It made it challenging for the models to deal with long sentences. Attention is for

recall

Time step: 7

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



At time step 7, the attention mechanism enables the **decoder** to focus on the word "étudiant" ("student" in french) before it generates the English translation. This ability to amplify the signal from the relevant part of the input sequence makes attention models produce better results than models without attention.



An attention model differs from a classic sequence-to-sequence model in two main ways:

First, the **encoder** passes a lot more data to the **decoder**. Instead of passing the last hidden state of the encoding stage, the **encoder** passes *all* the **hidden states** to the **decoder**.

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Je

suis

étudiant



Summary:

1. Look at the set of encoder **hidden states** it received – each **encoder hidden states** is most associated with a certain word in the input sentence
2. Give each **hidden states** a score (let's ignore how the scoring is done for now)
3. Multiply each **hidden states** by its softmaxed score, thus amplifying **hidden**

Attention at time step 4

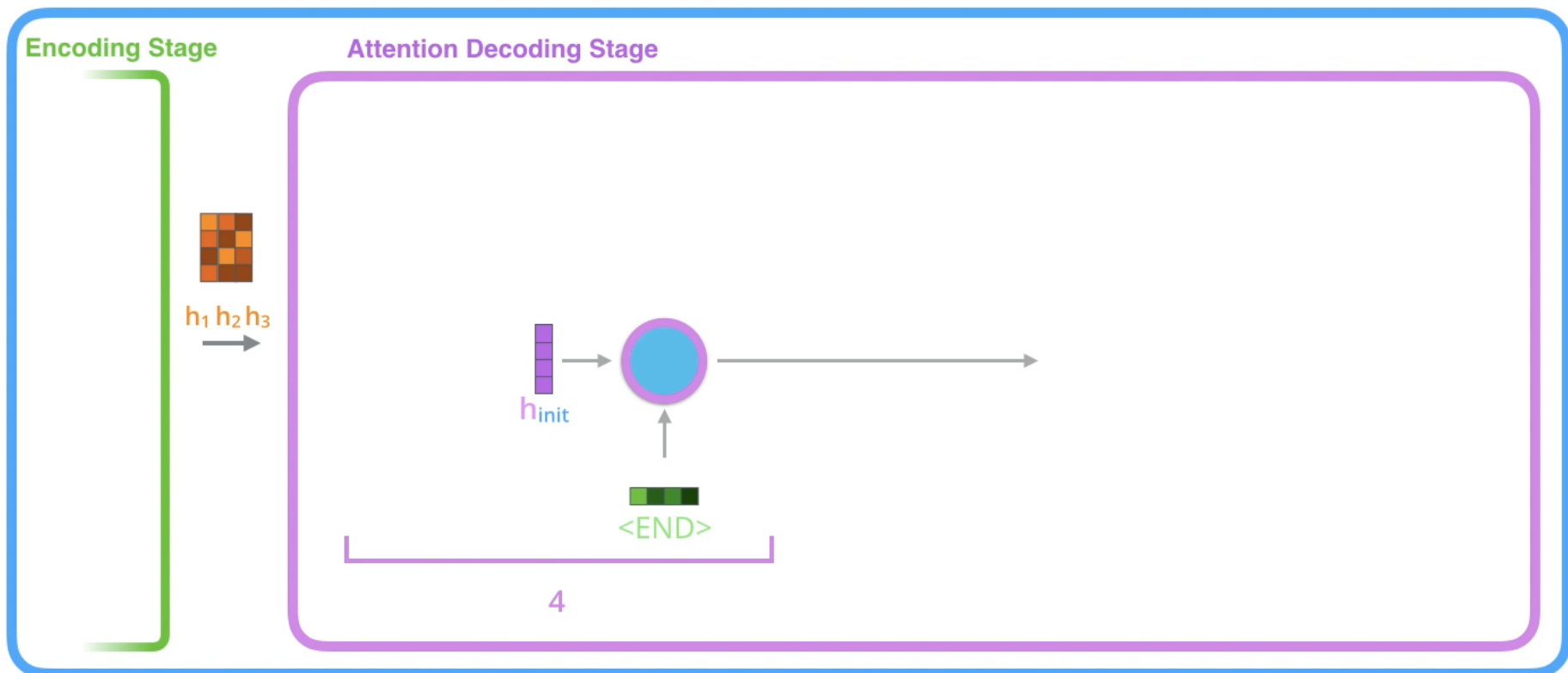


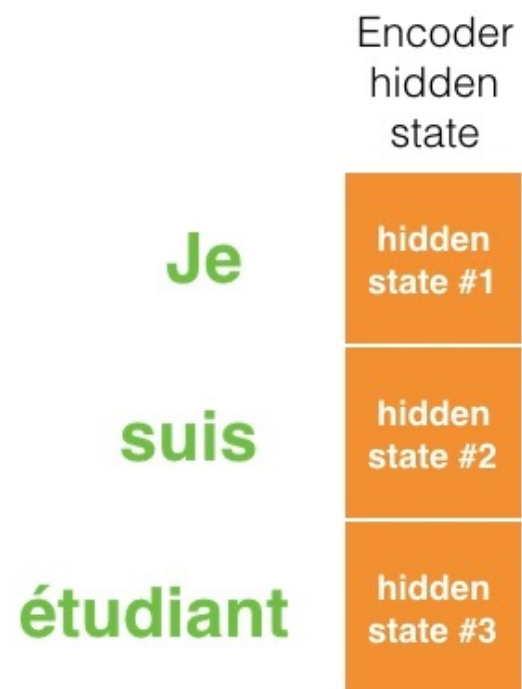
1. The attention decoder RNN takes in the embedding of the **<END>** token, and an **initial decoder hidden state**.
2. The RNN processes its inputs, producing an output and a **new hidden state** vector (**h4**). The output is discarded.
3. Attention Step: We use the **encoder hidden states** and the **h4** vector to calculate a context vector (**C4**) for this time step.
4. We concatenate **h4** and **C4** into one vector.
5. We pass this vector through a **feedforward neural network** (one trained jointly with the model).
6. The **output** of the feedforward neural networks indicates the output word of this time step.
7. Repeat for the next time steps

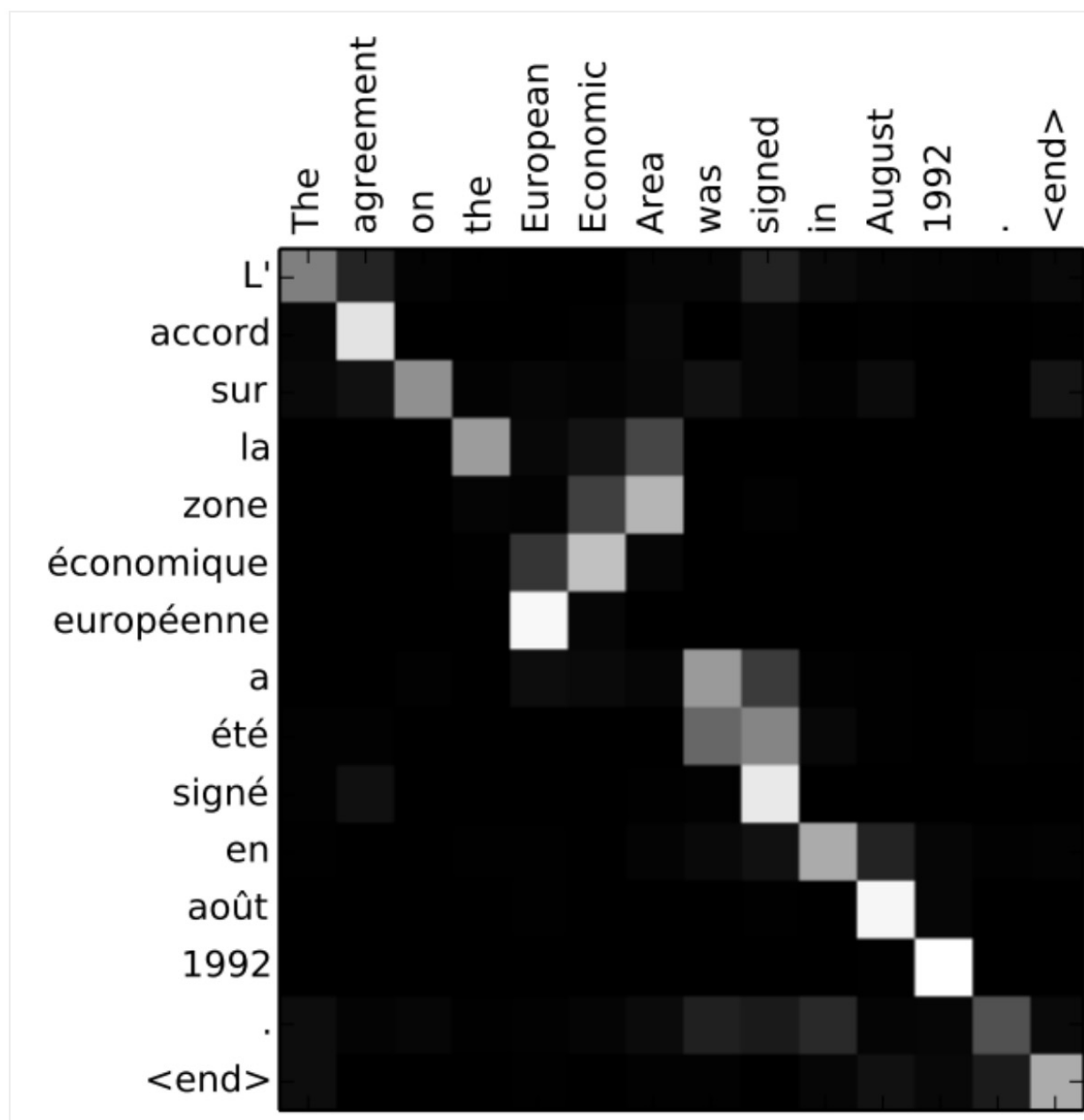


Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION





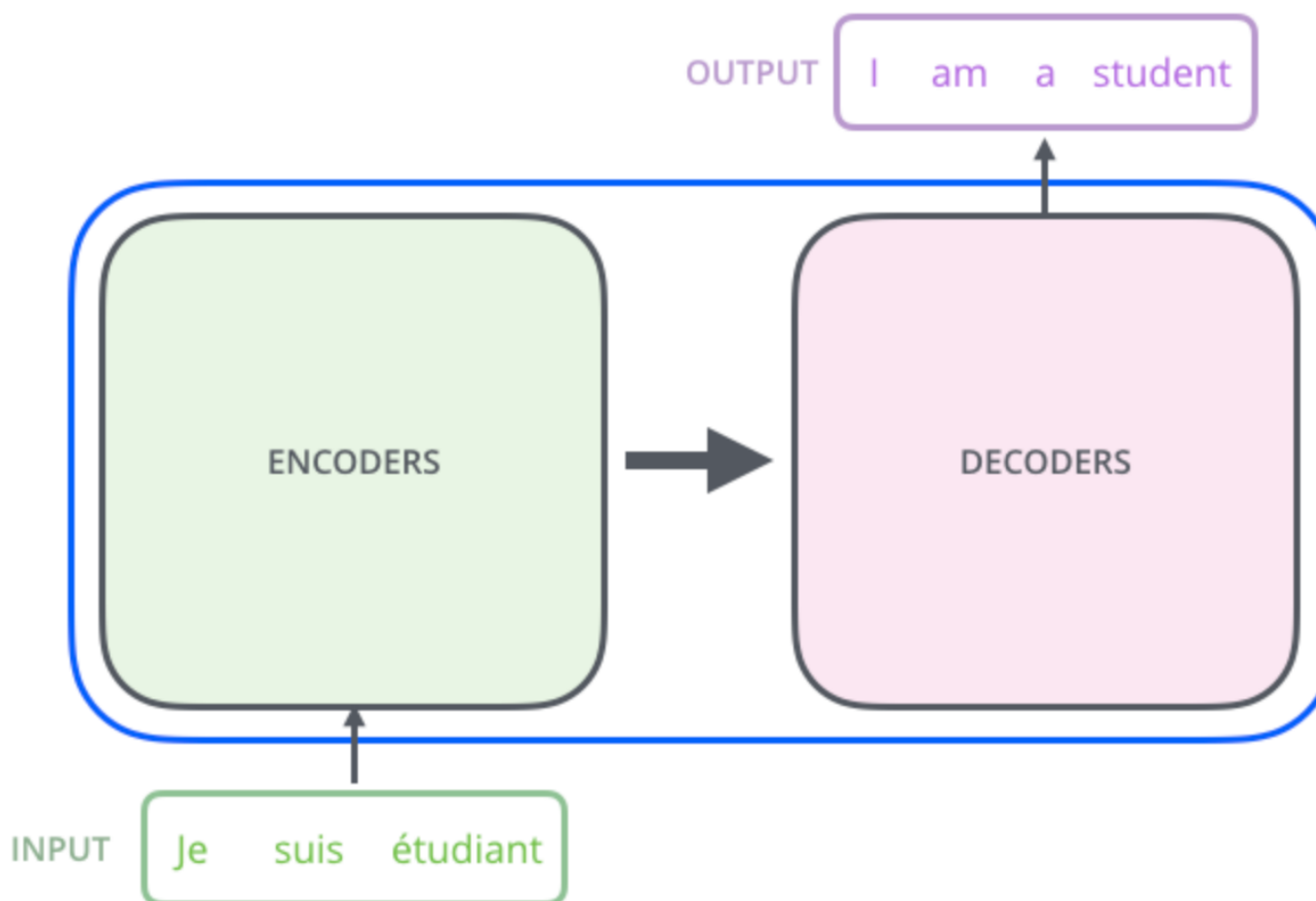


Transformer

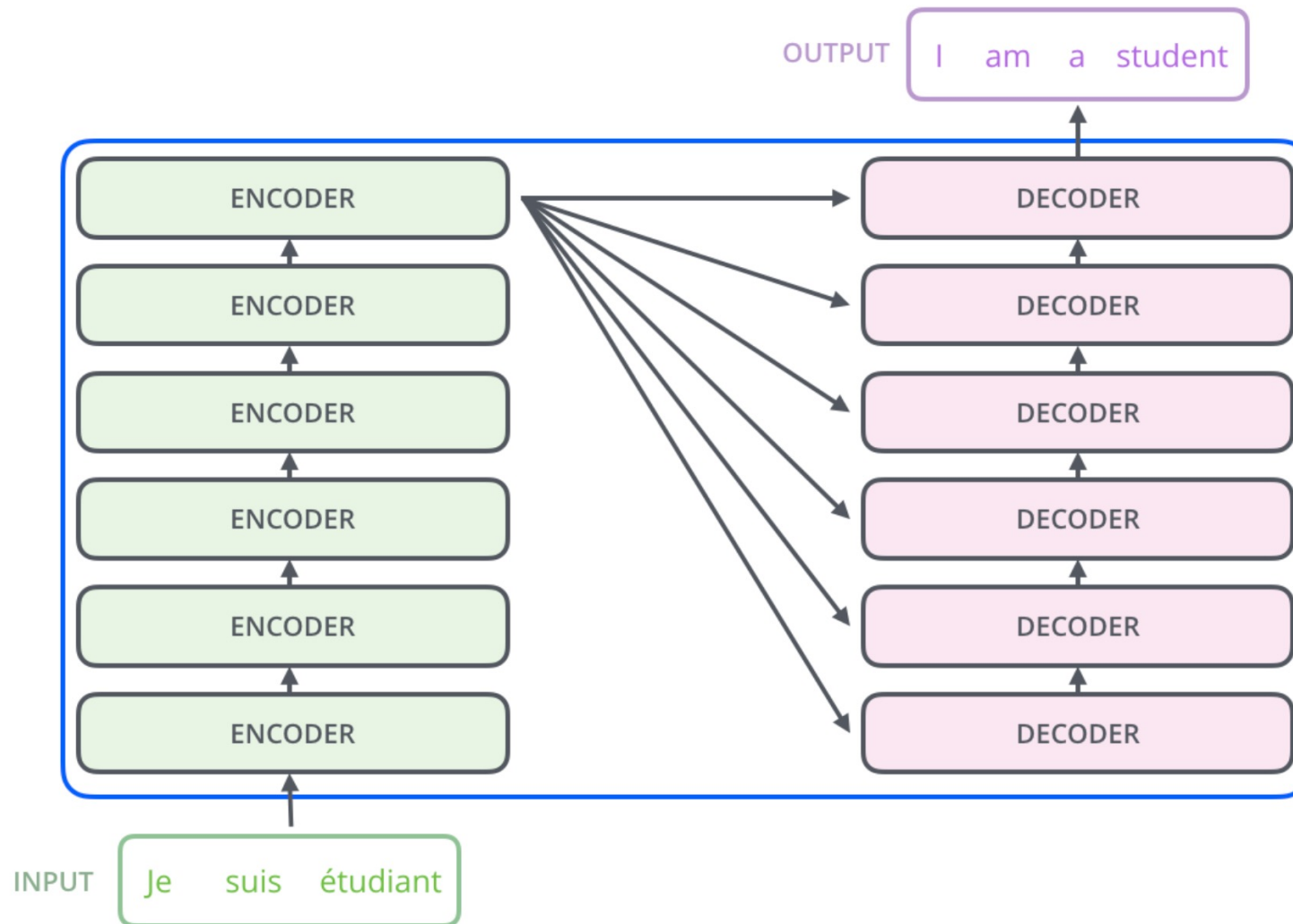
Transformer



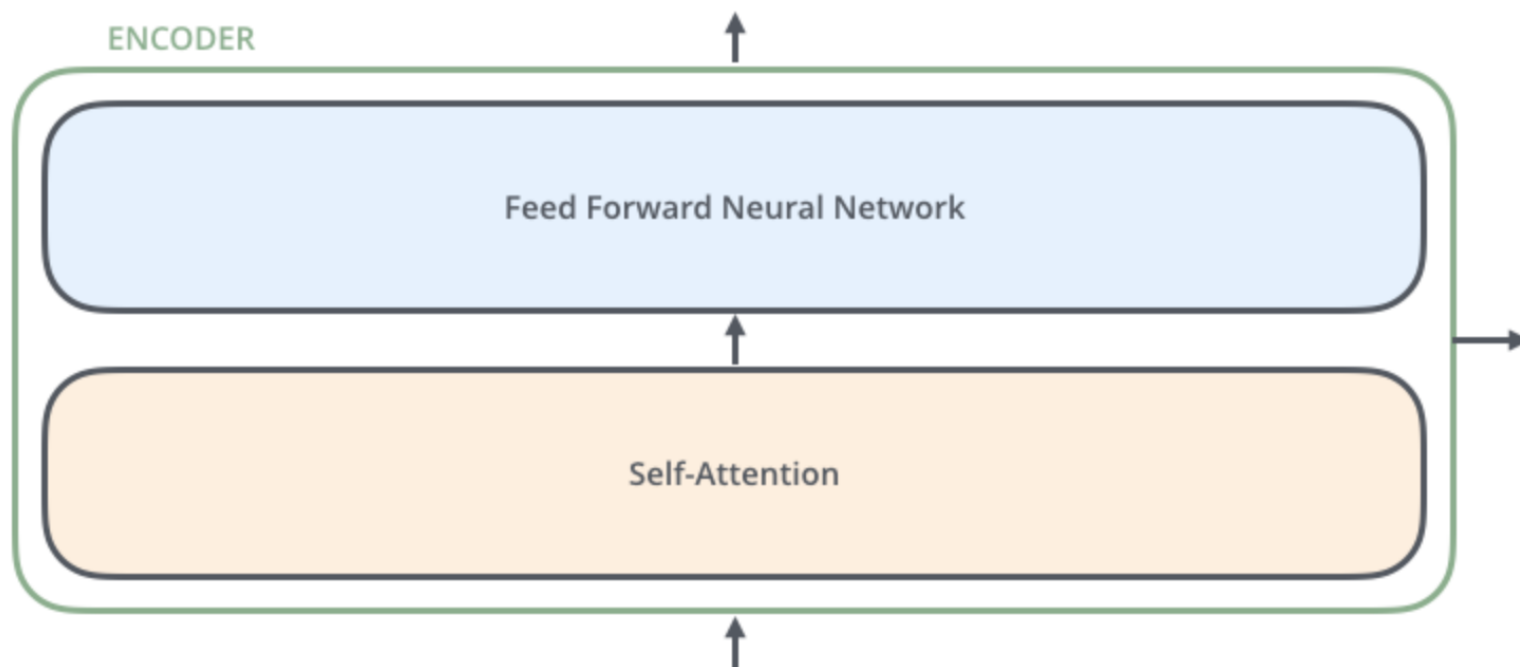
Encoder-Decoder



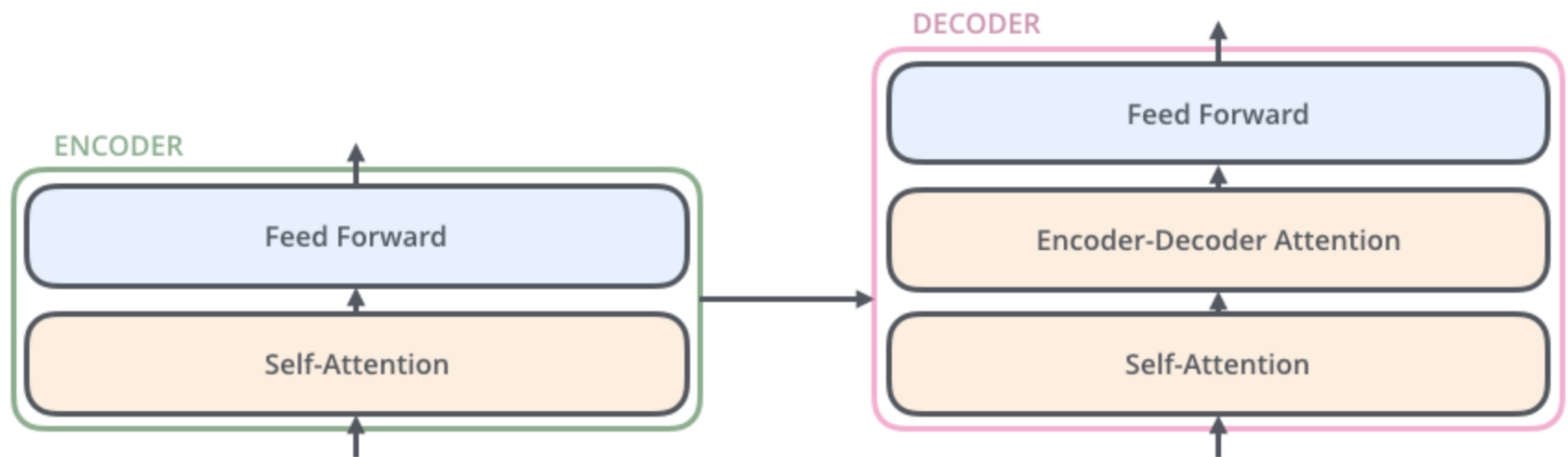
Encoders - Decoders



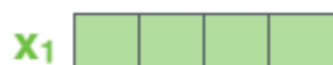
Encoder



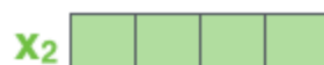
Decoder



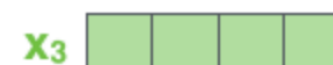
Input Words as Embeddings



Je



suis

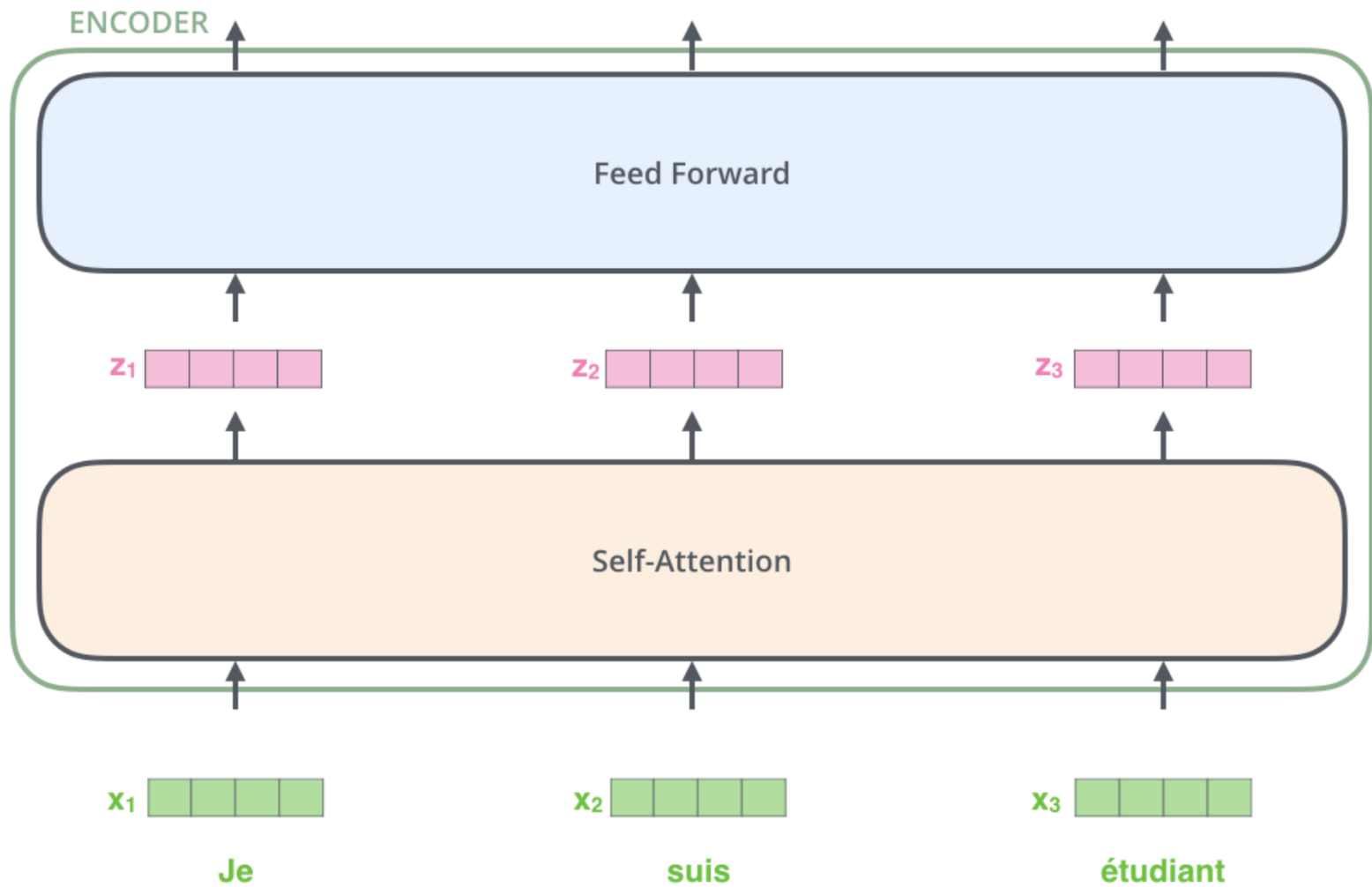


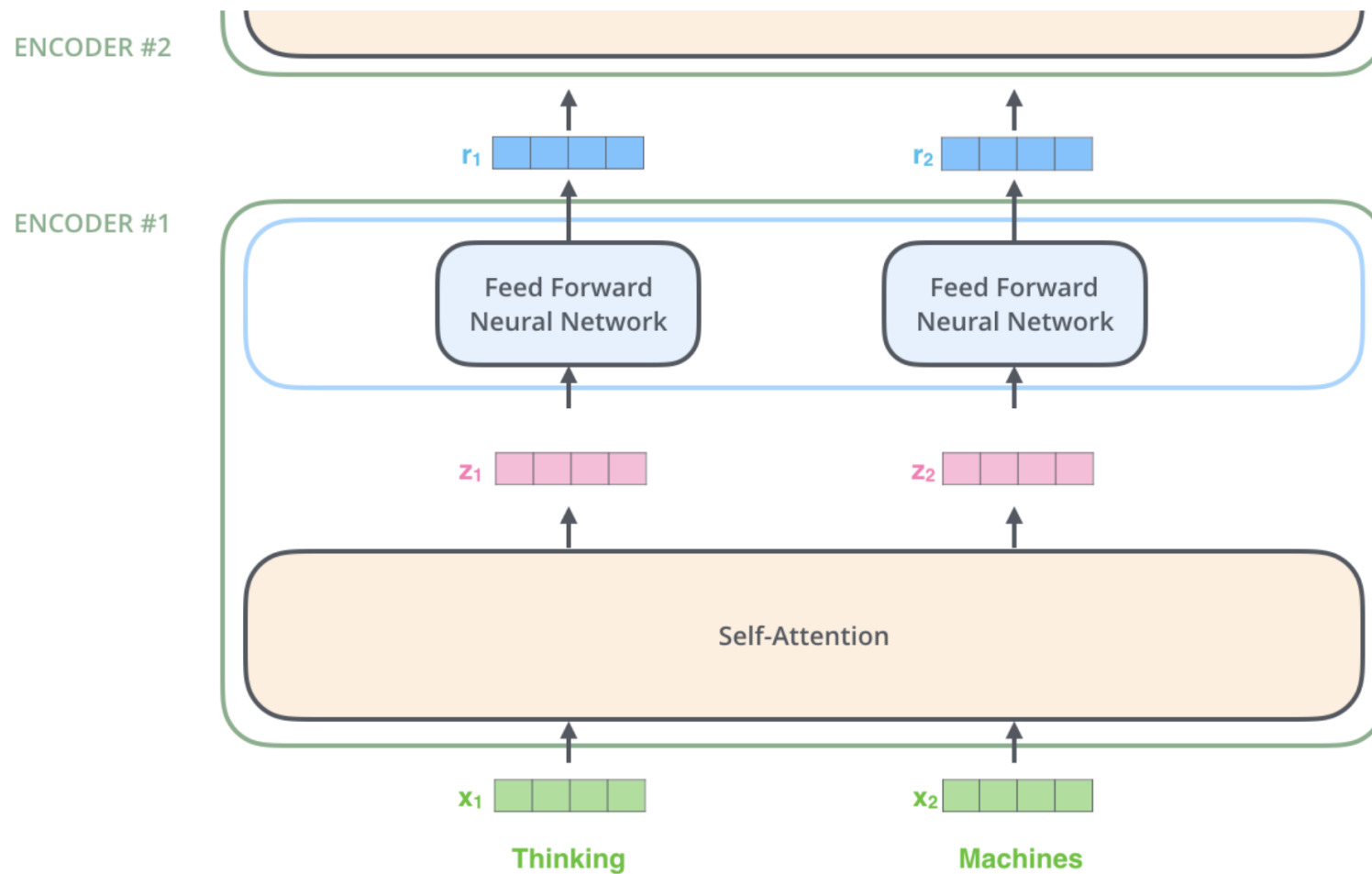
étudiant

Each word is embedded into a vector of size 512. We'll represent those vectors with these simple boxes.



Encoder



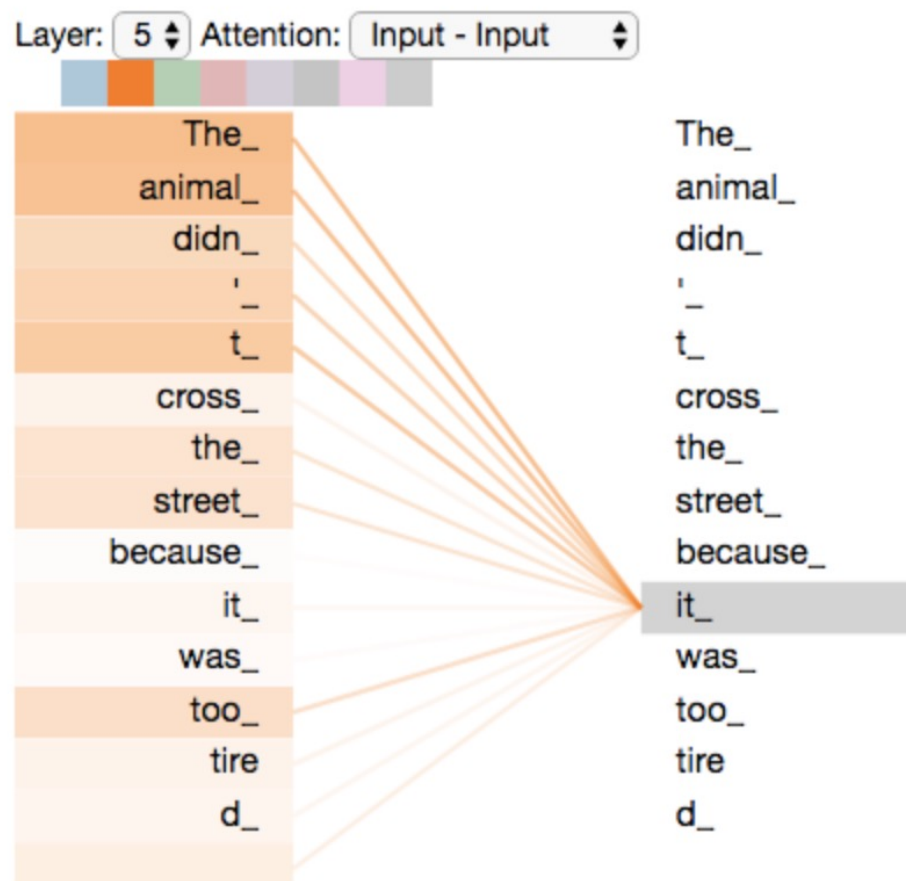


The word at each position passes through a self-attention process. Then, they each pass through a feed-forward neural network -- the exact same network with each vector flowing through it separately.

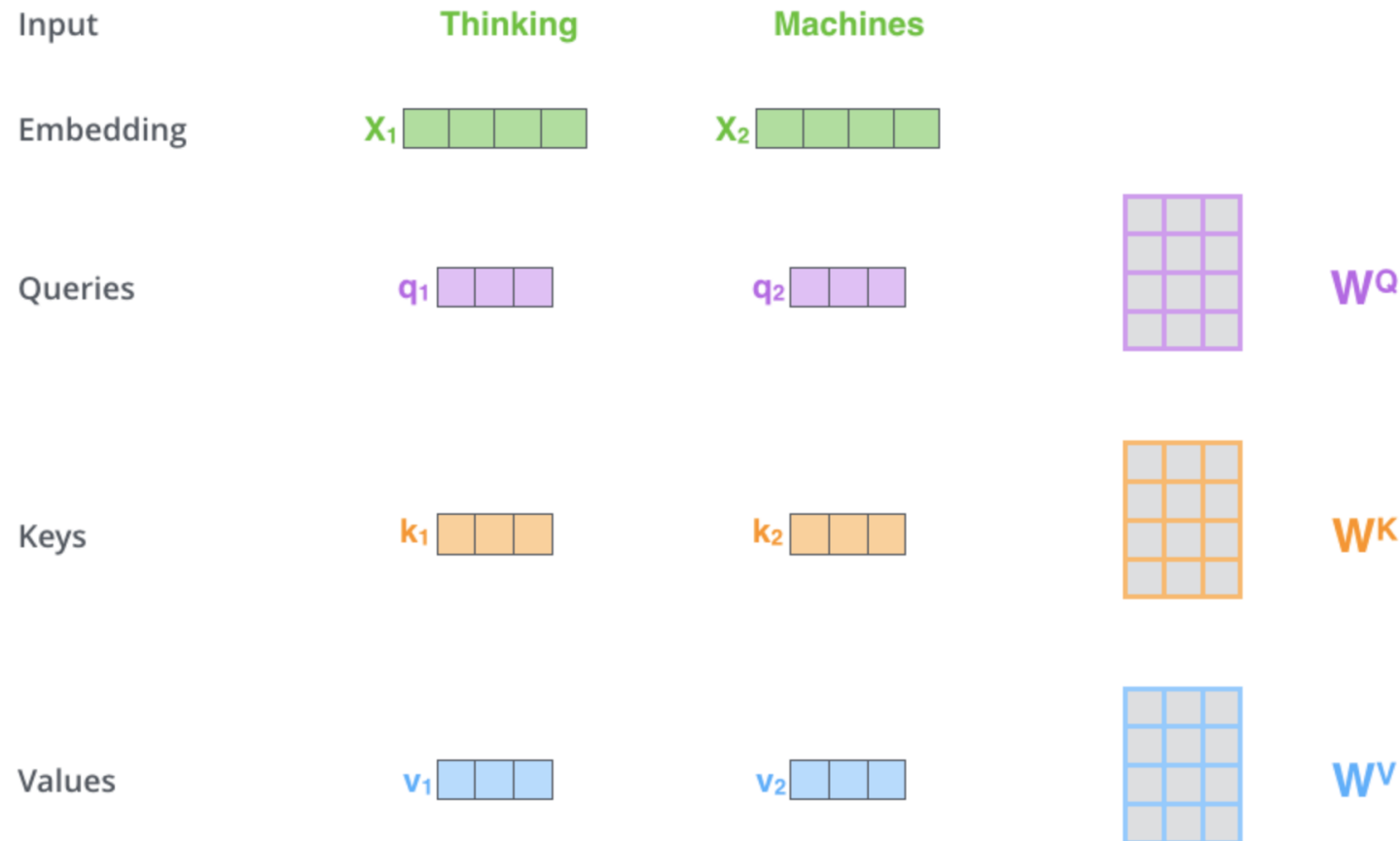


Attention

"The animal didn't cross the street because it was too tired"



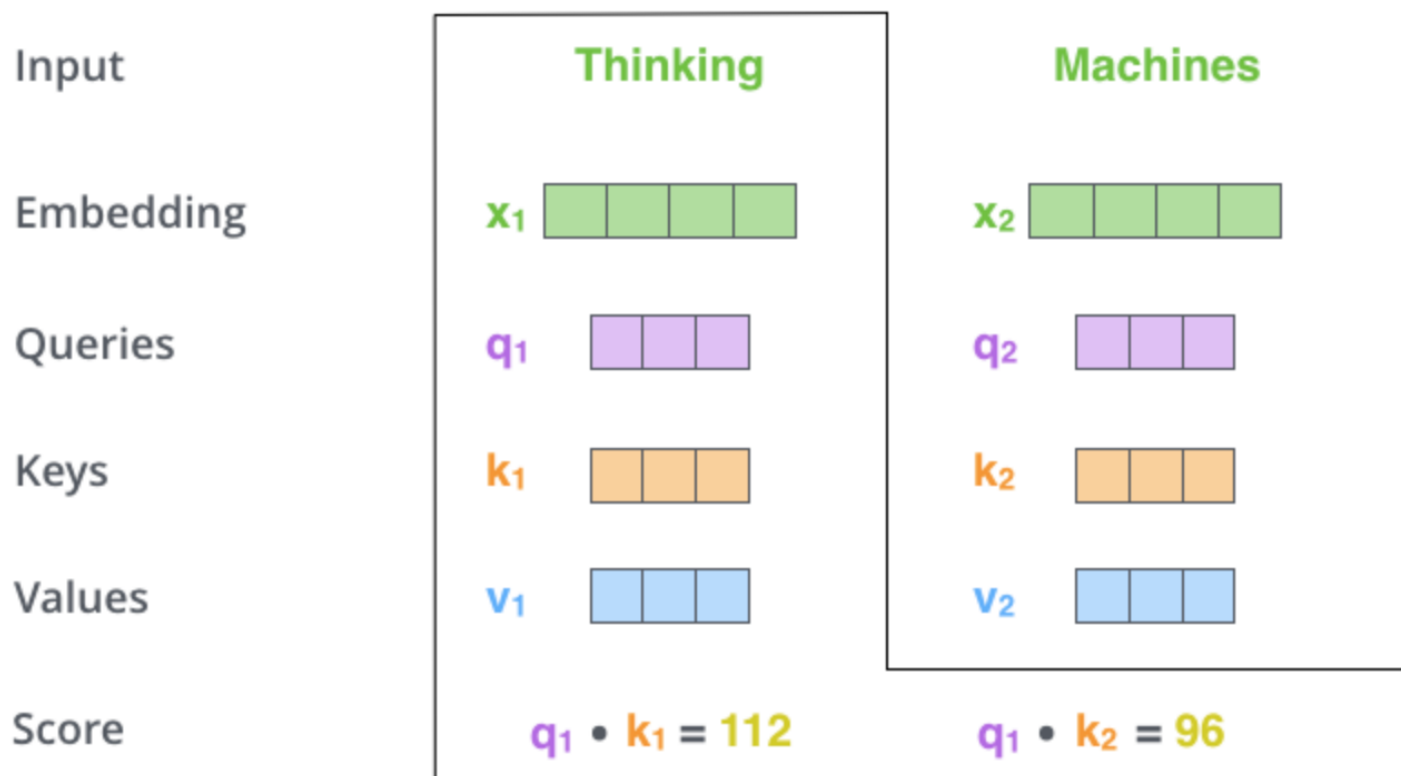
Keys, Values, Queries



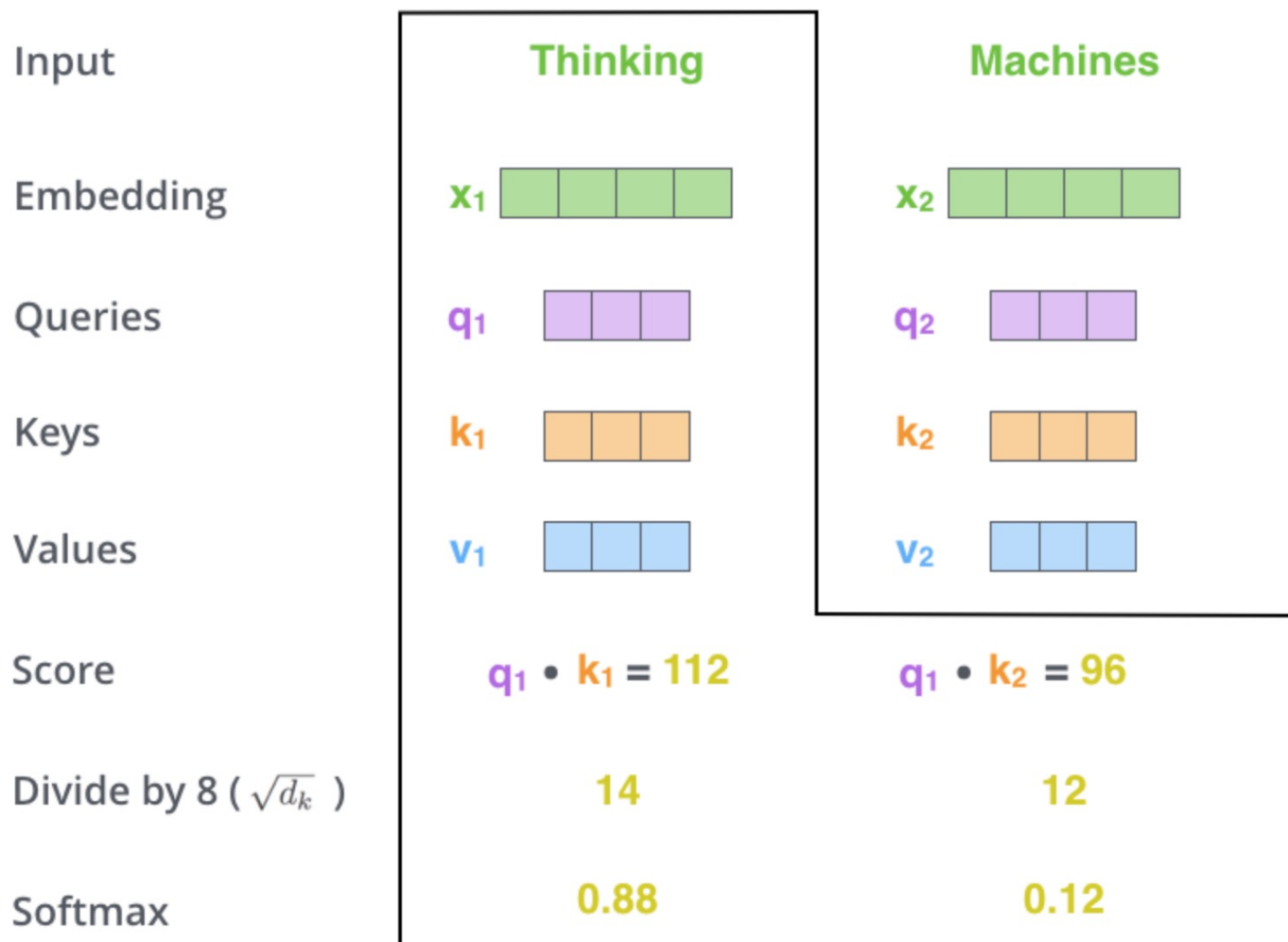
Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.



Score



Softmax

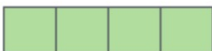


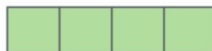
Input

Thinking

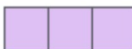
Machines

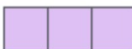
Embedding

x_1 

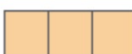
x_2 

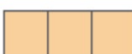
Queries

q_1 

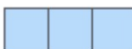
q_2 

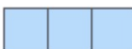
Keys

k_1 

k_2 

Values

v_1 

v_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

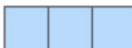
Softmax

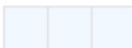
0.88

0.12

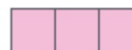
Softmax

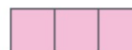
X
Value

v_1 

v_2 

Sum

z_1 

z_2 



K, V, Q



Z

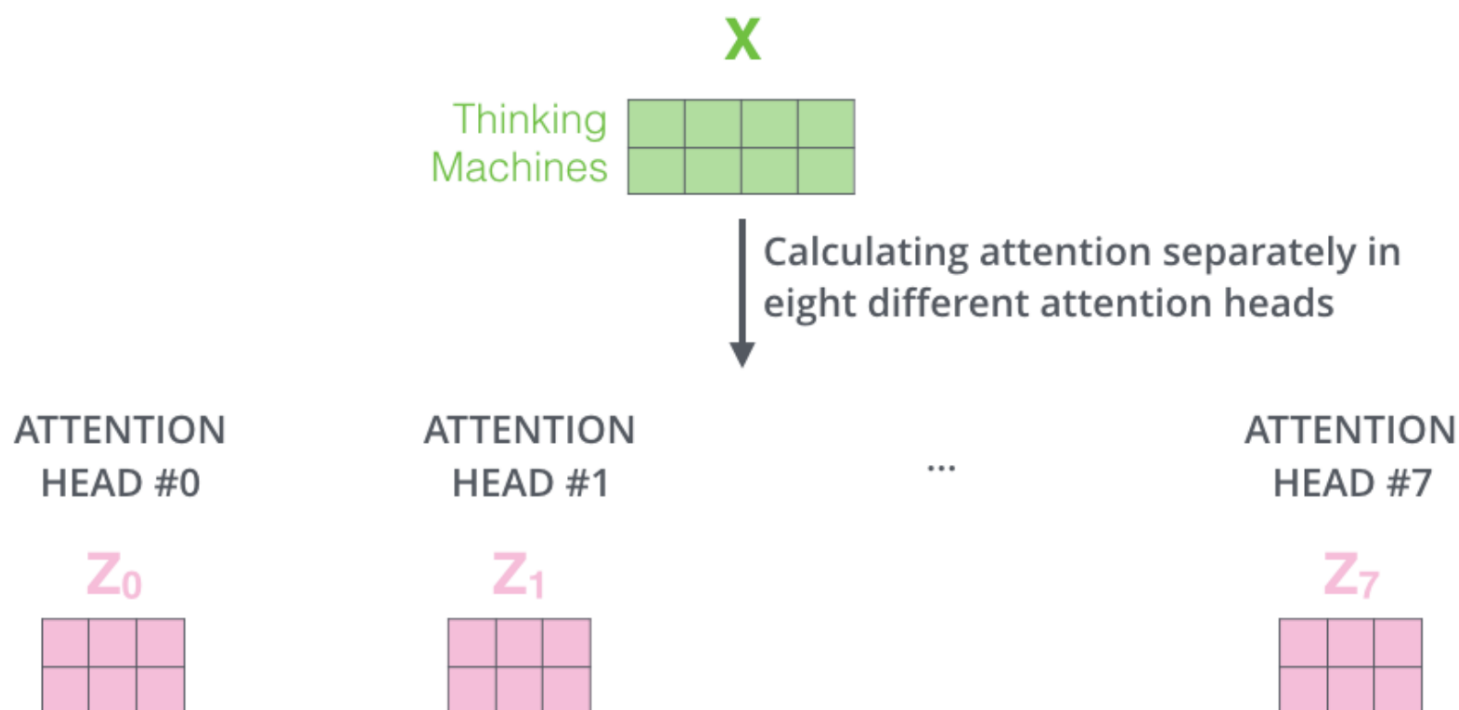
$$\text{softmax} \left(\frac{
 \begin{matrix} \textcolor{violet}{Q} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \times \begin{matrix} \textcolor{brown}{K}^T \\ \begin{array}{|c|c|} \hline & \\ \hline & \\ \hline & \\ \hline \end{array} \end{matrix} }{ \sqrt{d_k} } \right) \begin{matrix} \textcolor{blue}{V} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix} \\
 = \begin{matrix} \textcolor{pink}{Z} \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$



Attention Heads



Attention Heads



Aggregating Attention Heads

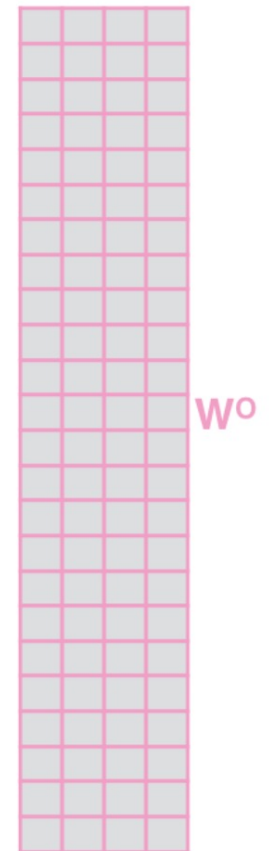
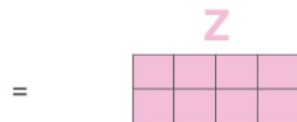
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

x

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Checkpoint

1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads.
We multiply X or R with weight matrices

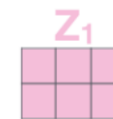
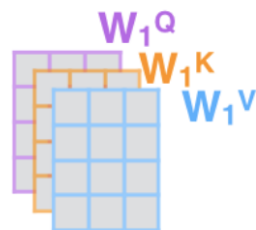
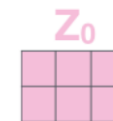
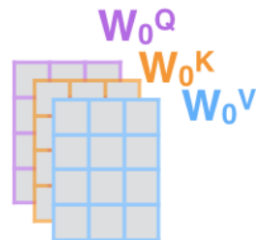
4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking Machines



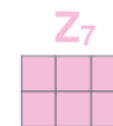
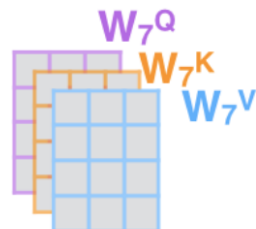
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

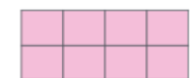
...



W^O

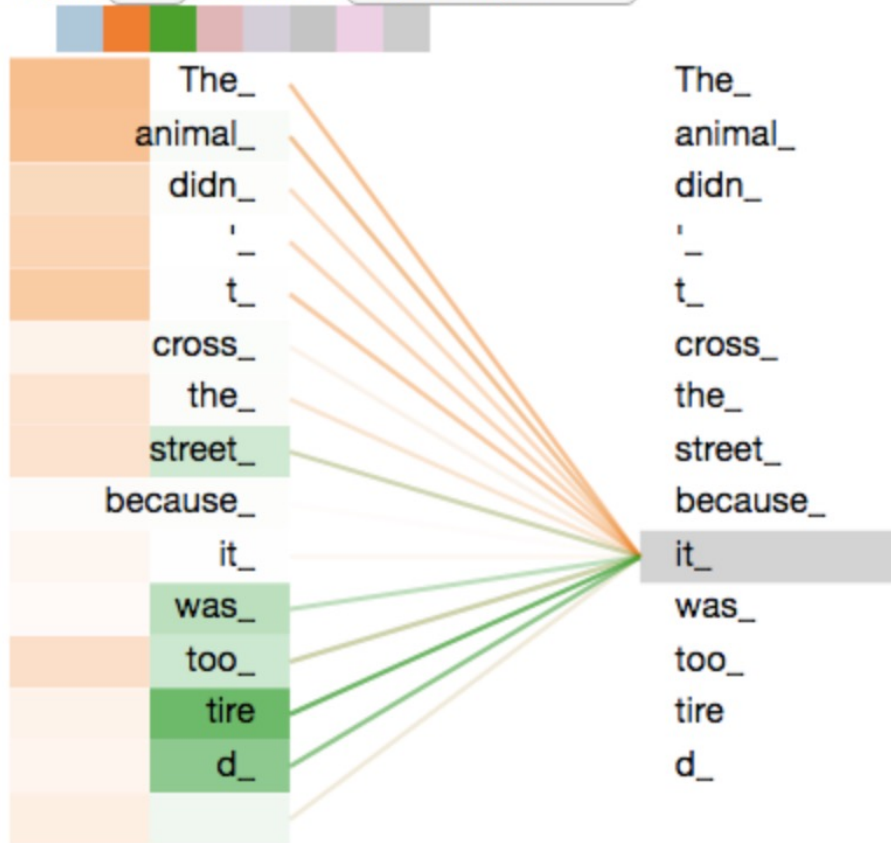


Z

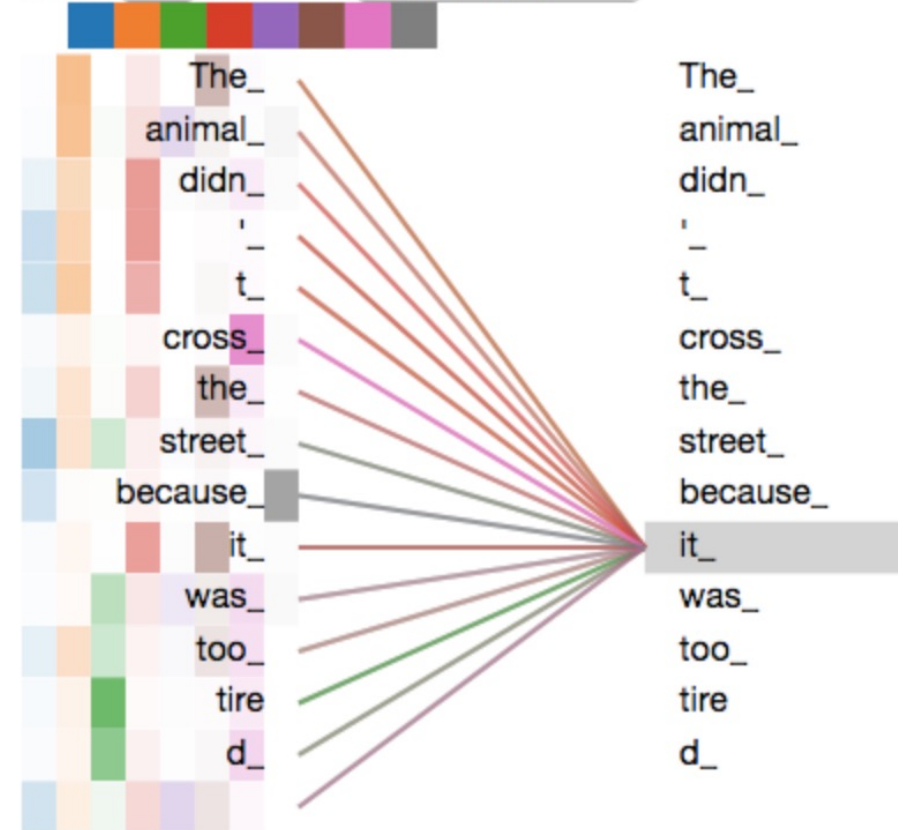


Multiple Attention Heads

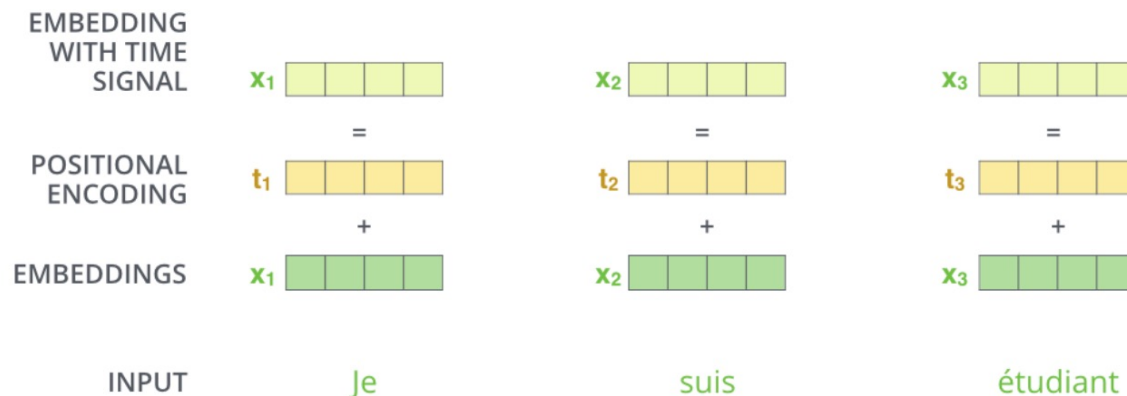
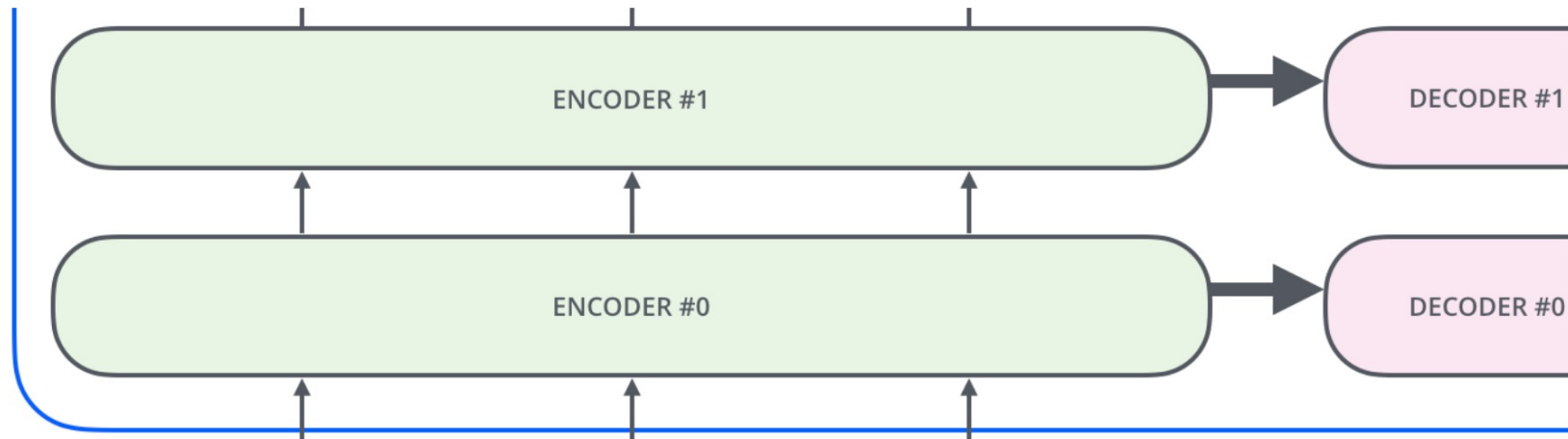
Layer: 5 ▾ Attention: Input - Input ▾



Layer: 5 ▾ Attention: Input - Input ▾



Positional Embeddings



To give the model a sense of the order of the words, we add positional encoding vectors -- the values of which follow a specific pattern.



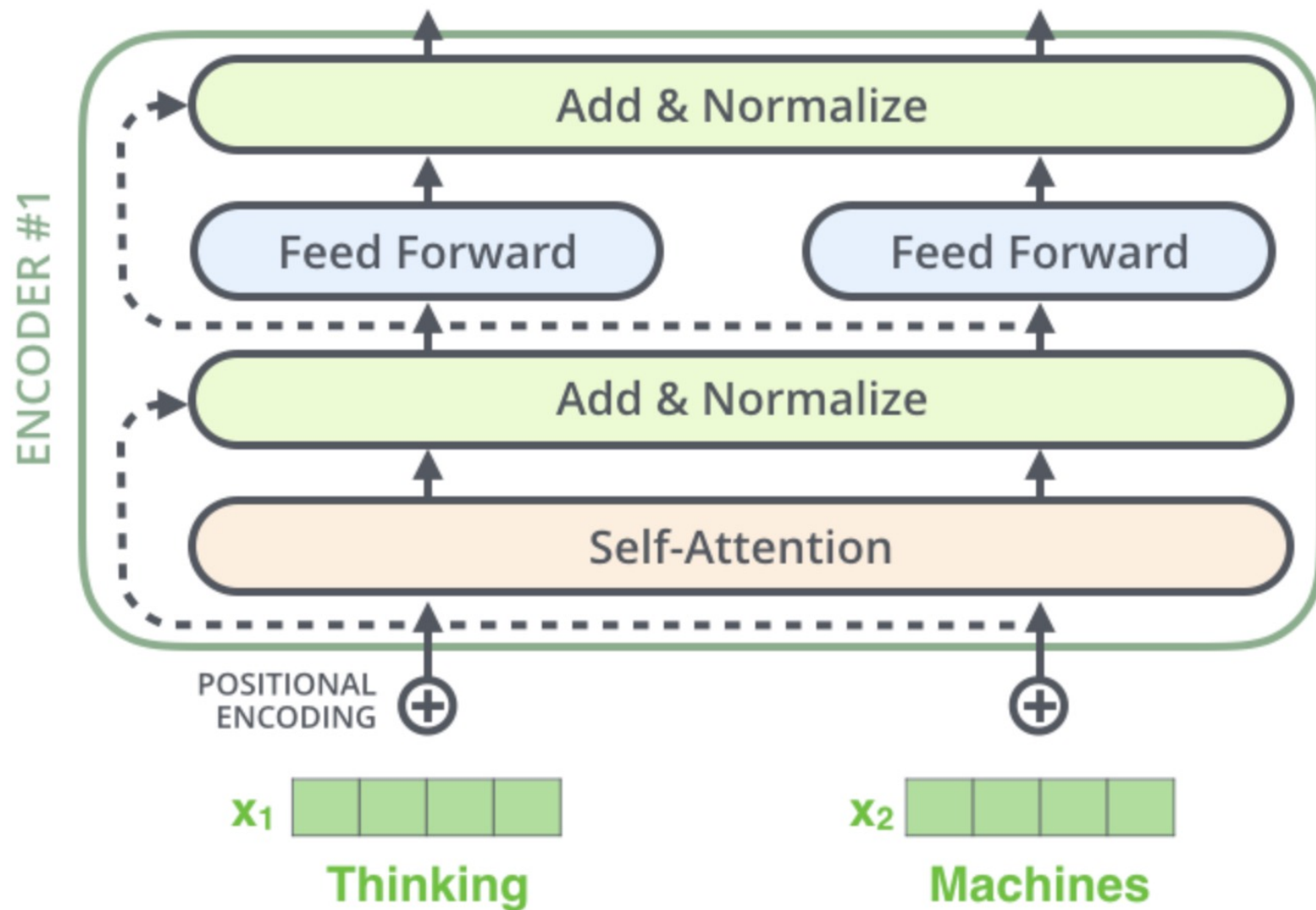
Positional Embeddings

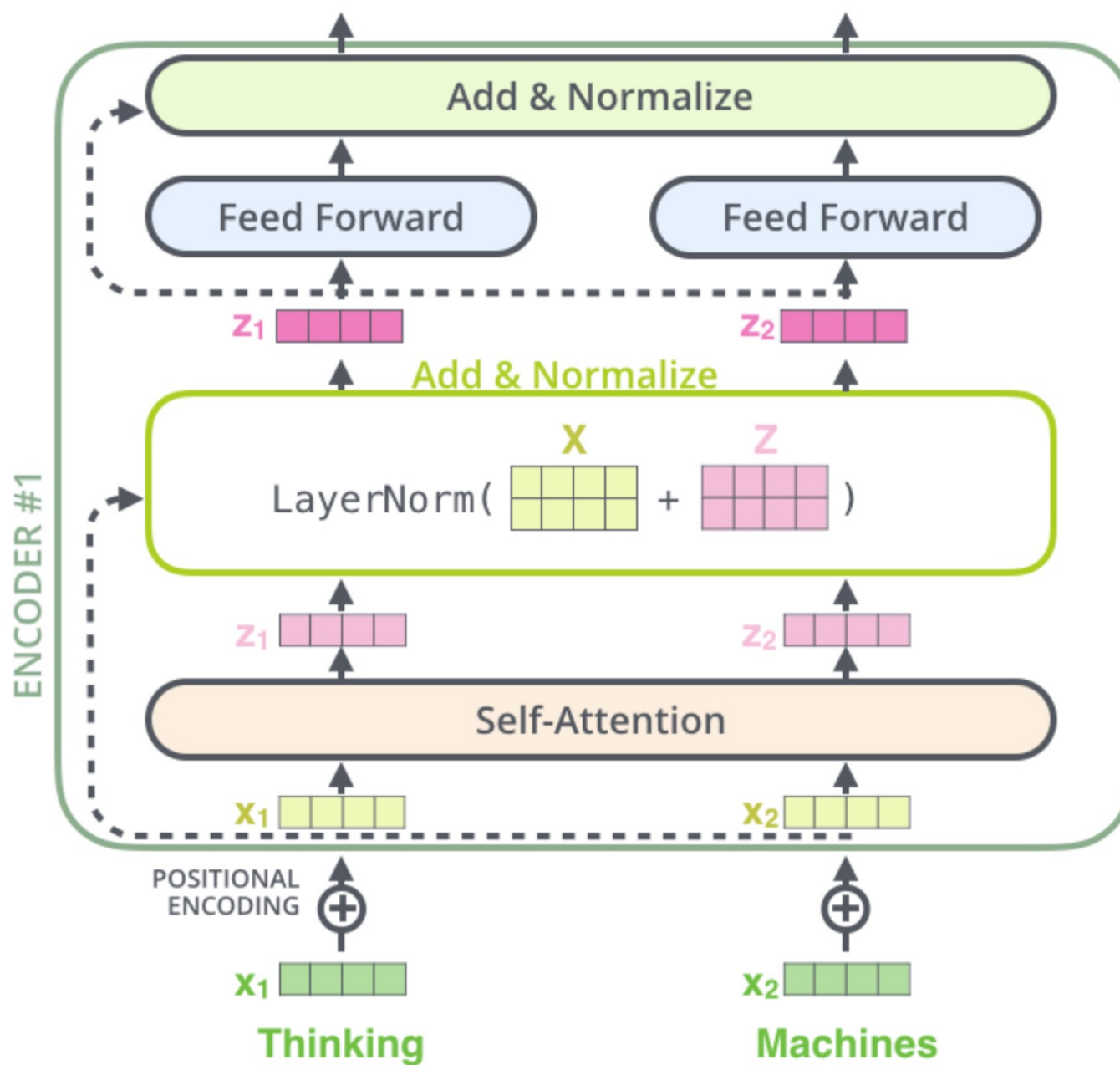


A real example of positional encoding with a toy embedding size of 4

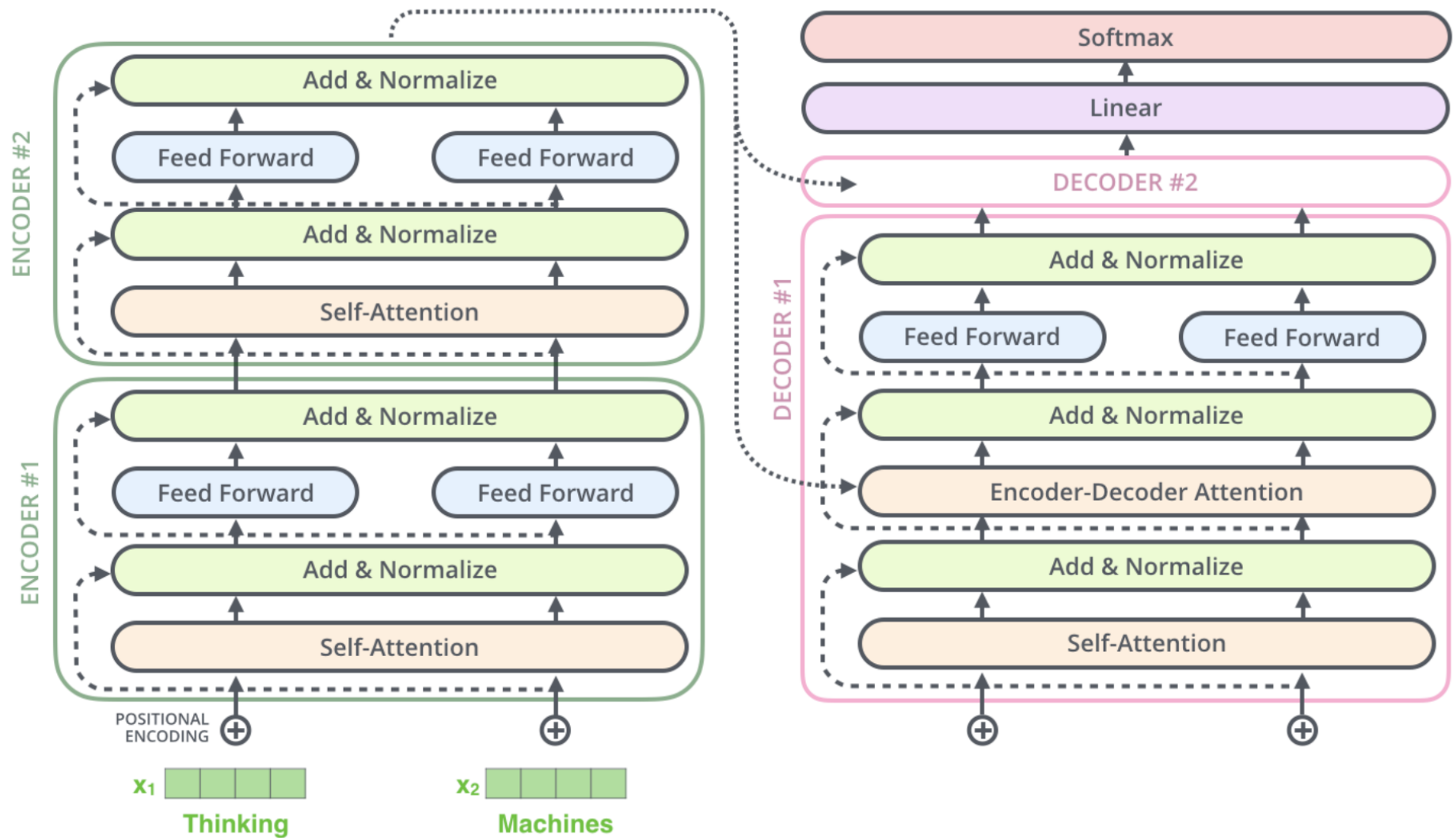


Encoder Revisited





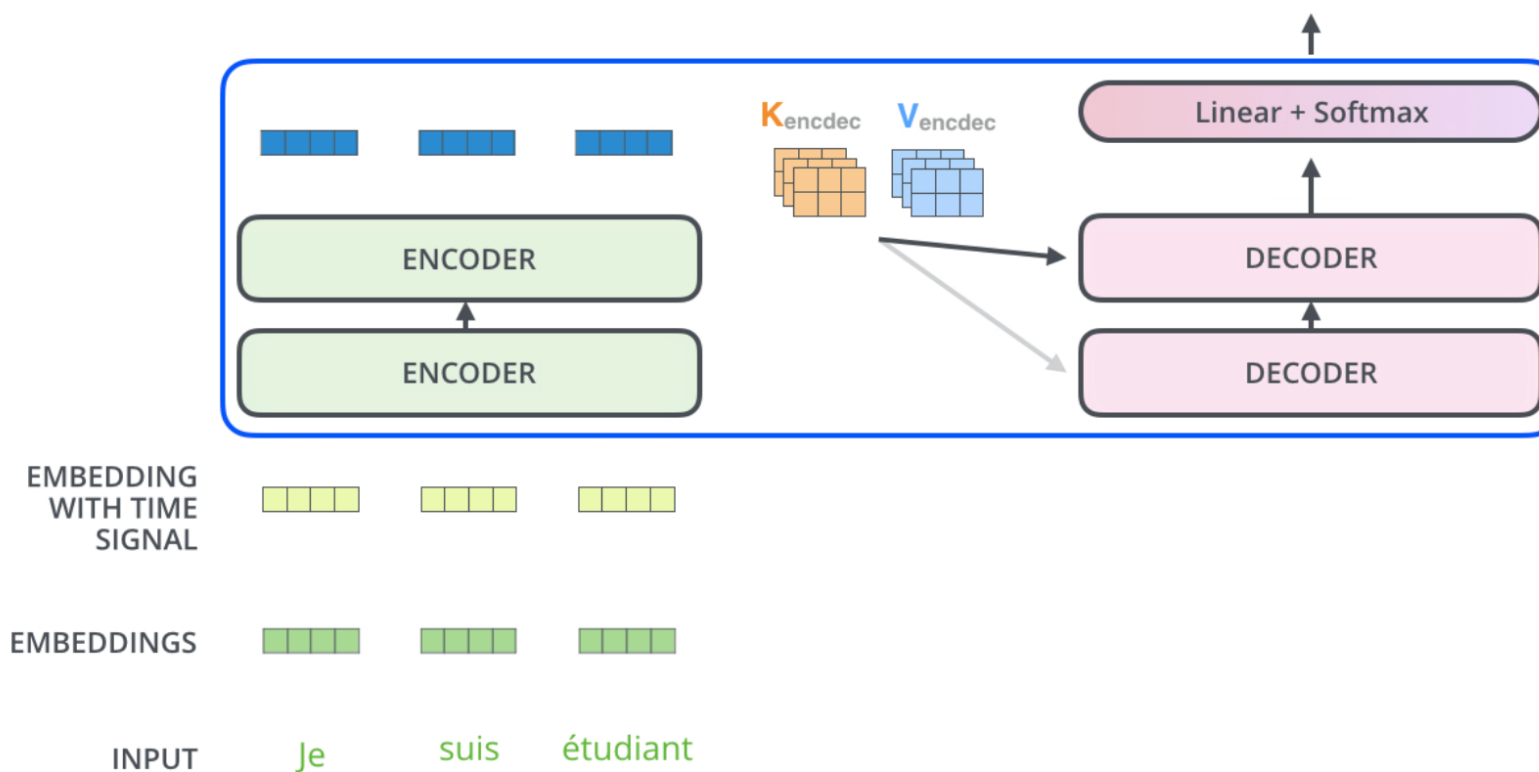
Encoder-Decoder



Decoding

Decoding time step: 1 2 3 4 5 6

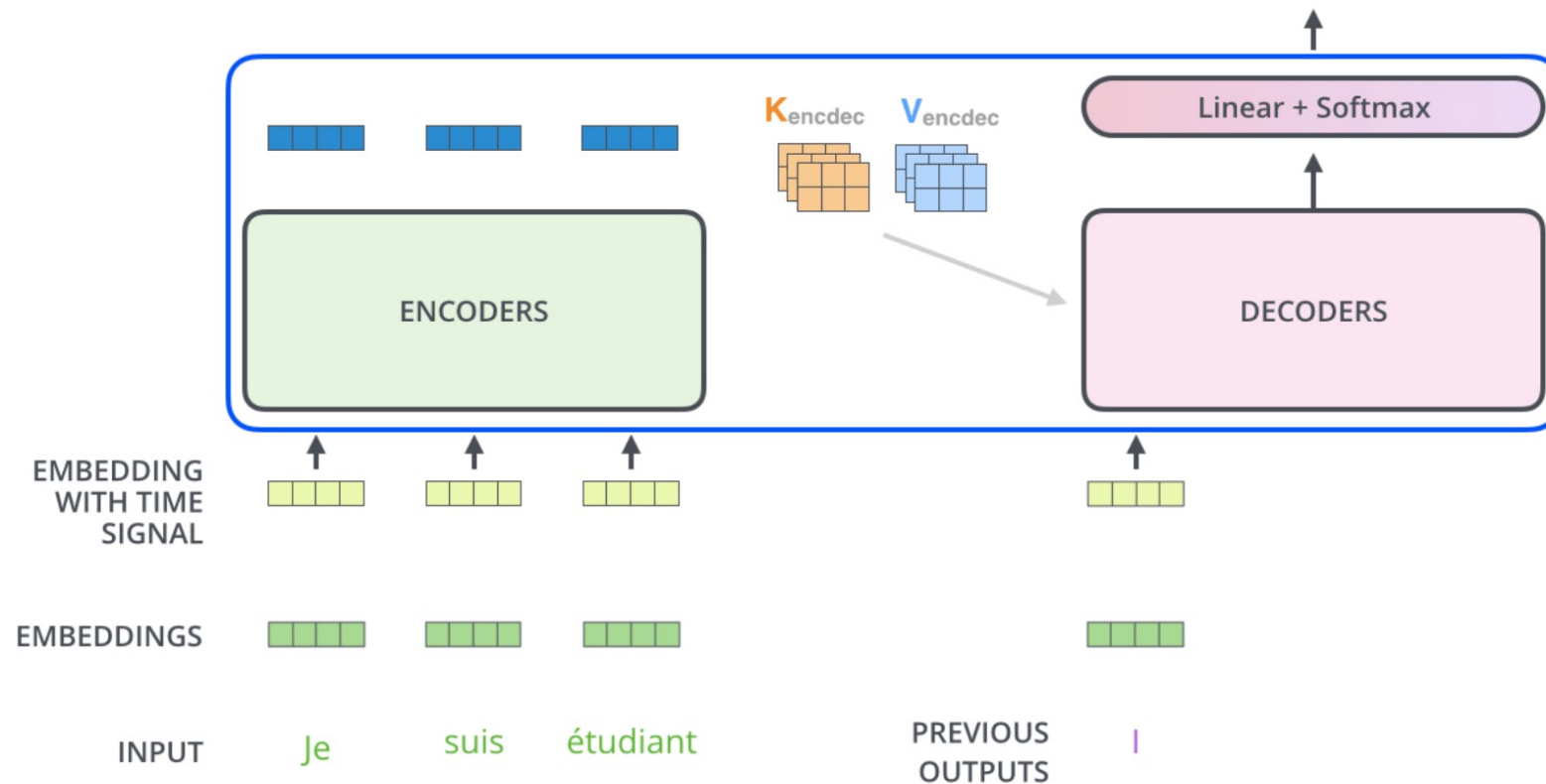
OUTPUT |



Decoding

Decoding time step: 1 2 3 4 5 6

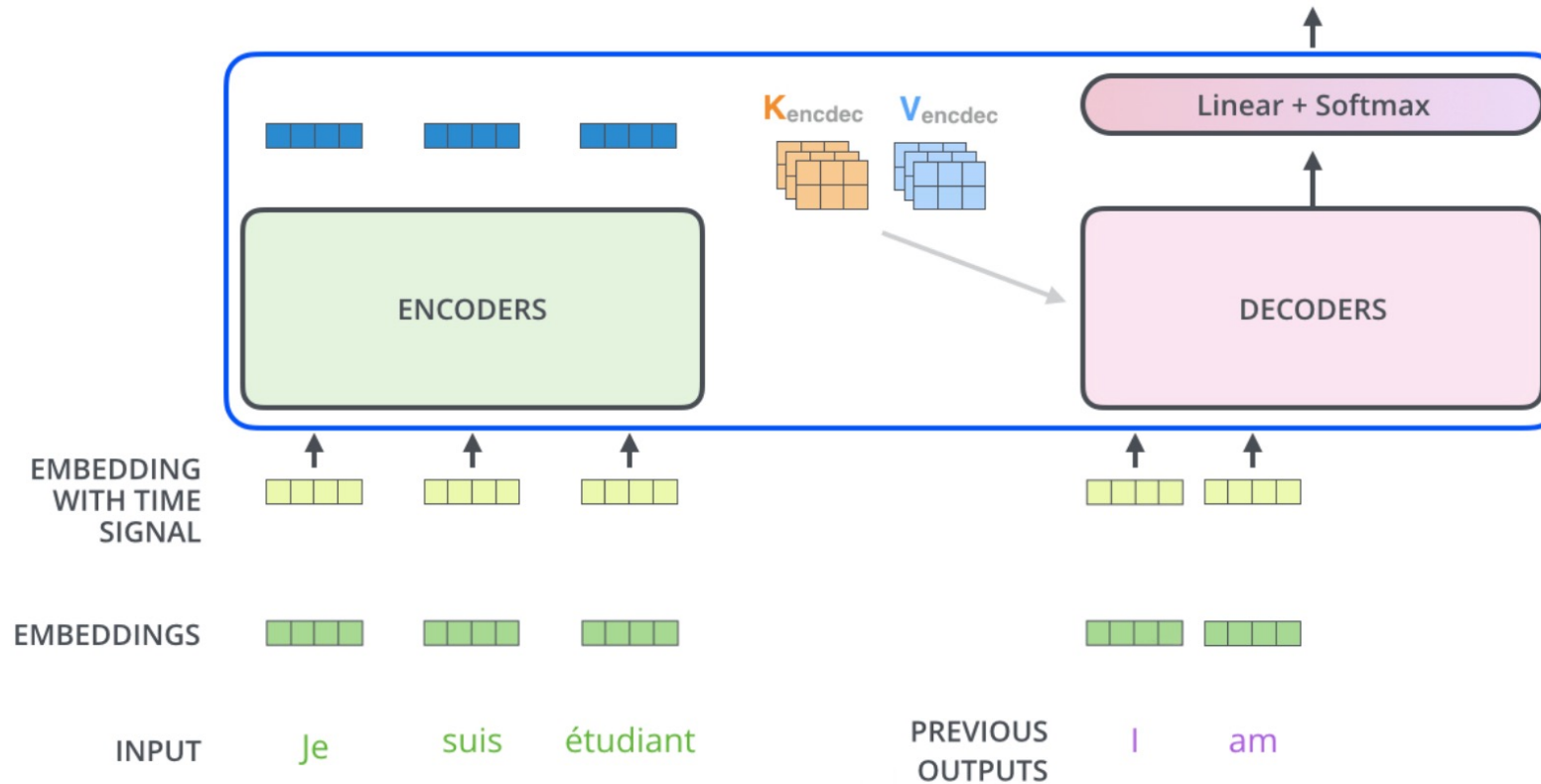
OUTPUT |



Decoding

Decoding time step: 1 2 **3** 4 5 6

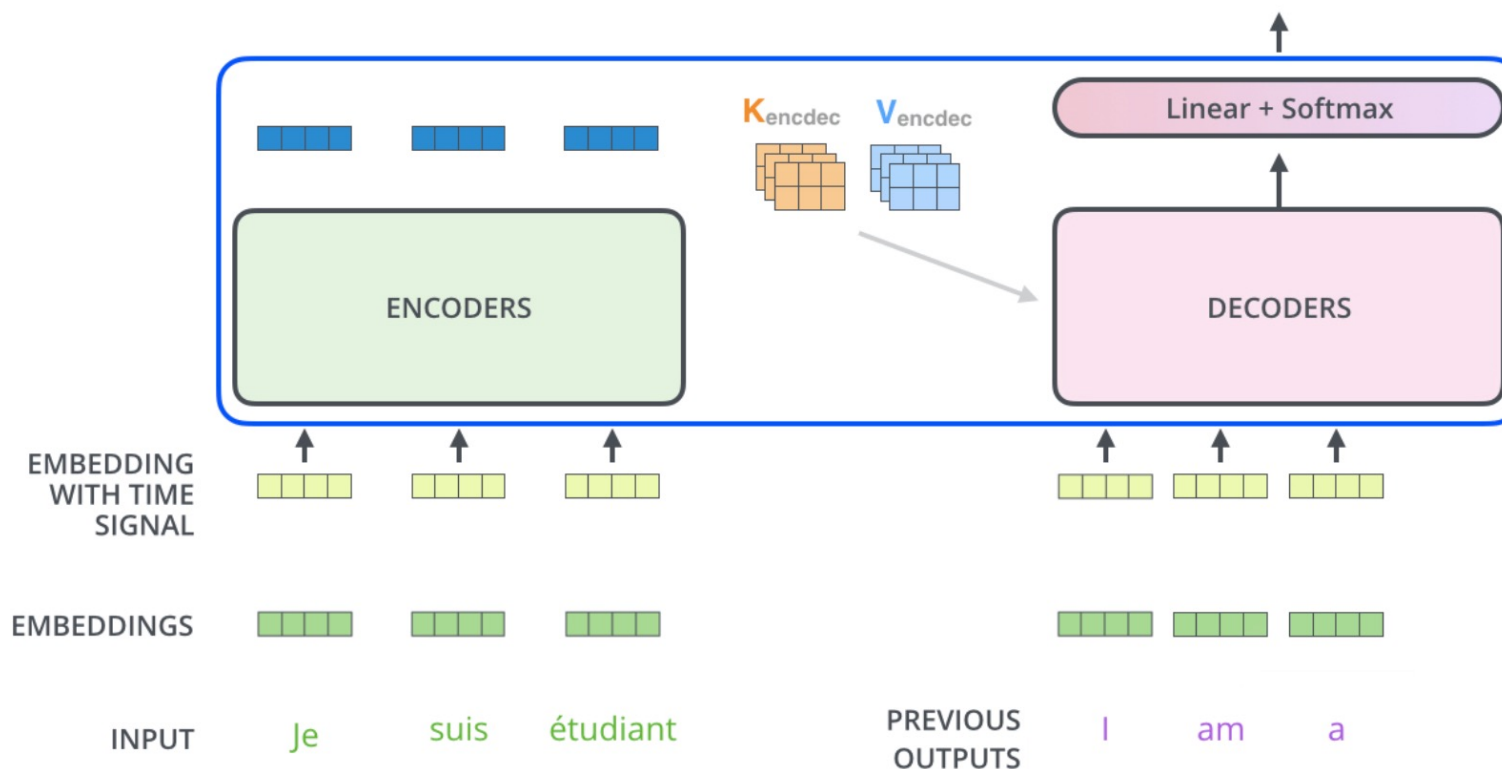
OUTPUT | am a



Decoding

Decoding time step: 1 2 3 4 5 6

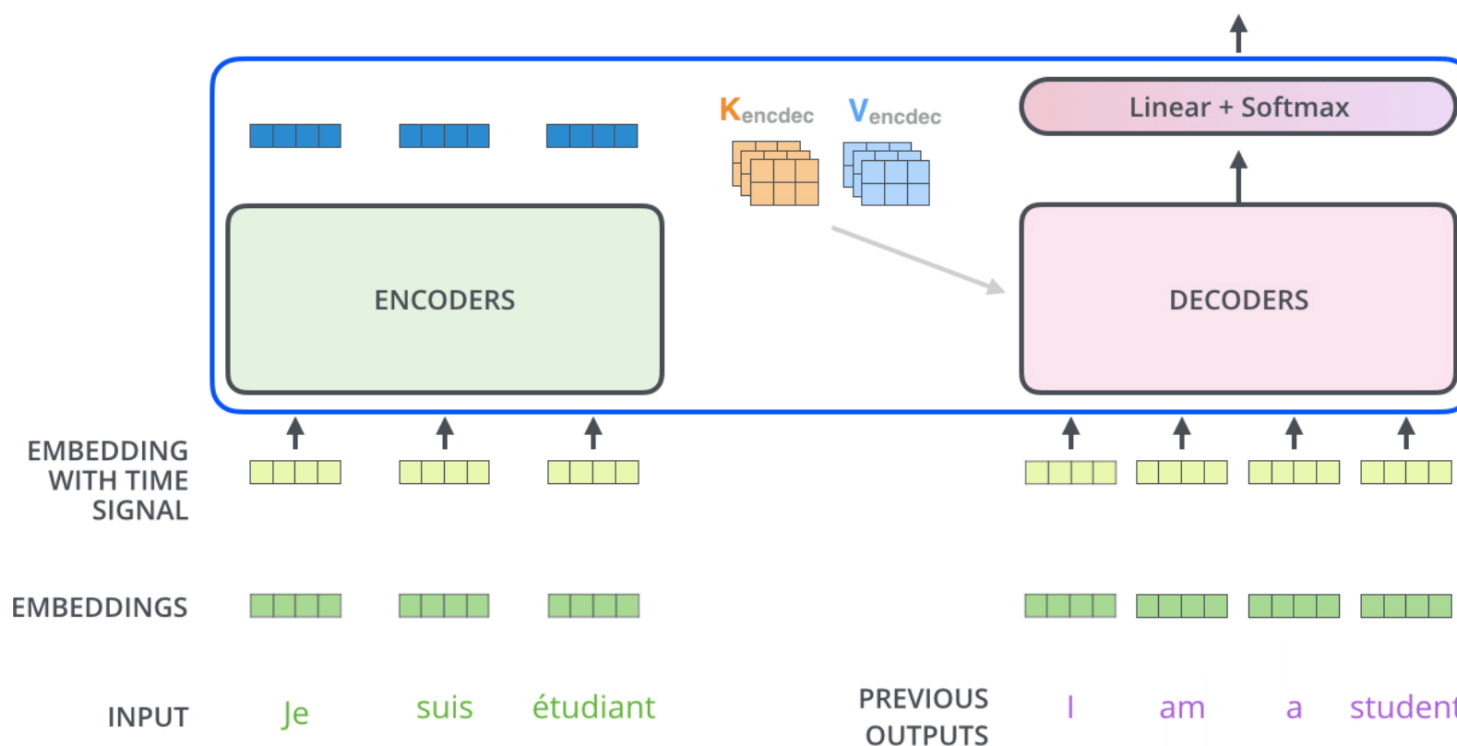
OUTPUT I am a student



Decoding

Decoding time step: 1 2 3 4 5 6

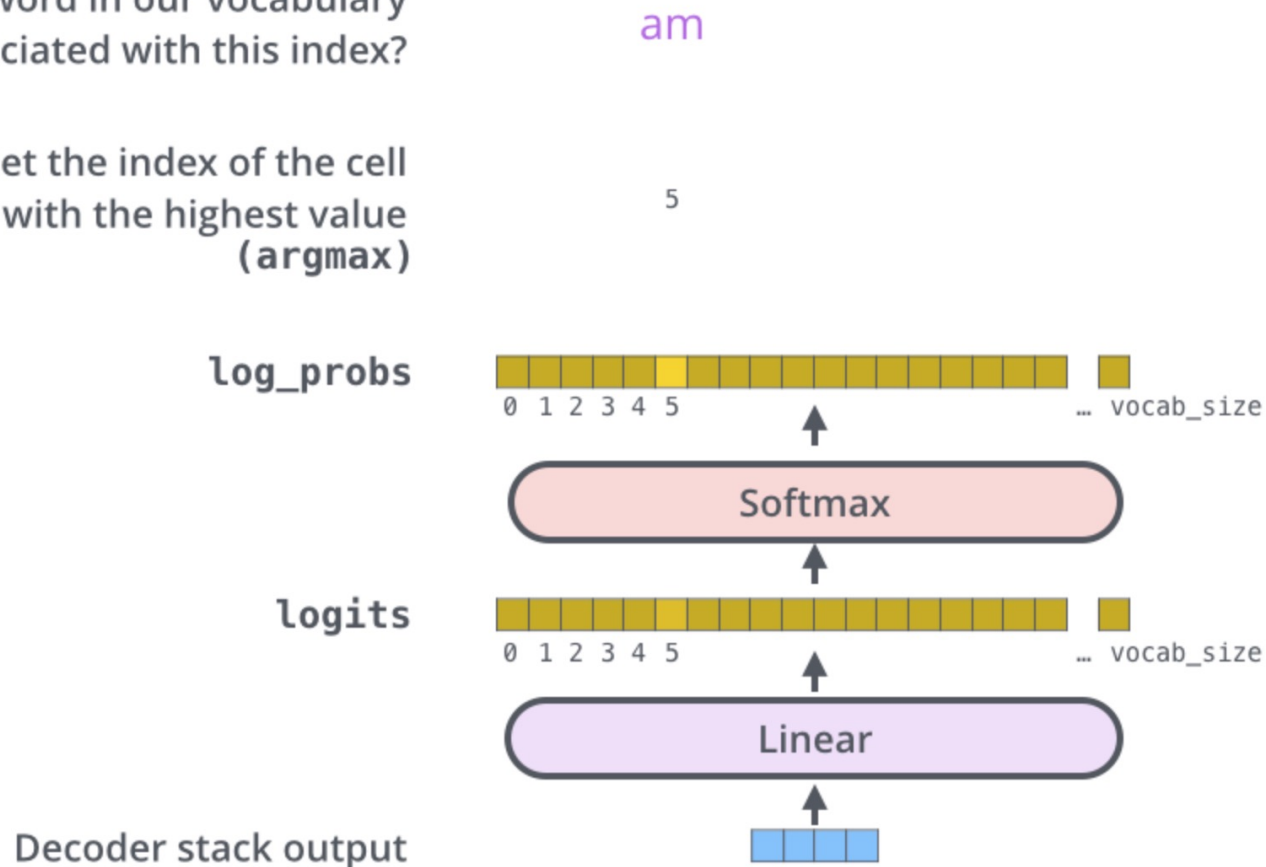
OUTPUT I am a student <end of sentence>



Decoder Output

Which word in our vocabulary is associated with this index?

Get the index of the cell with the highest value (argmax)



This figure starts from the bottom with the vector produced as the output of the decoder stack. It is then turned into an output word.



Positional OHE

Output Vocabulary						
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

The output vocabulary of our model is created in the preprocessing phase before we even begin training.

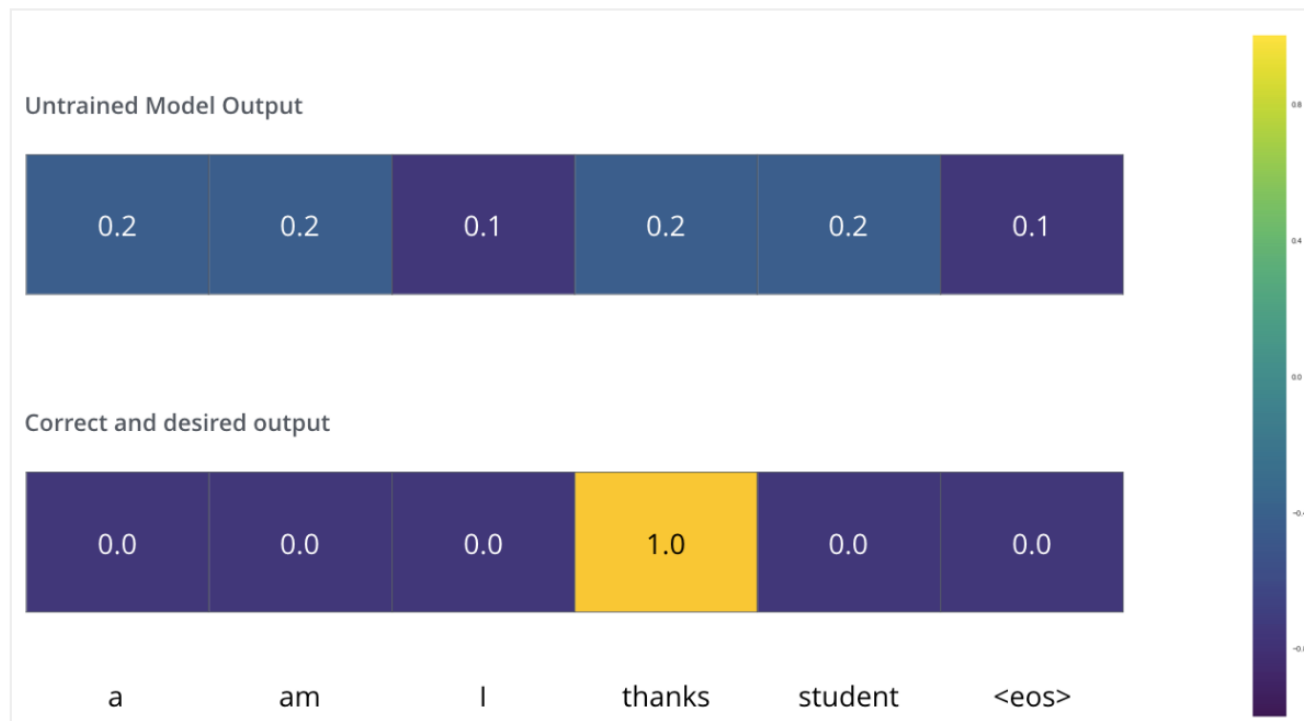
Output Vocabulary						
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5

One-hot encoding of the word "am"						
0.0	1.0	0.0	0.0	0.0	0.0	0.0

Example: one-hot encoding of our output vocabulary



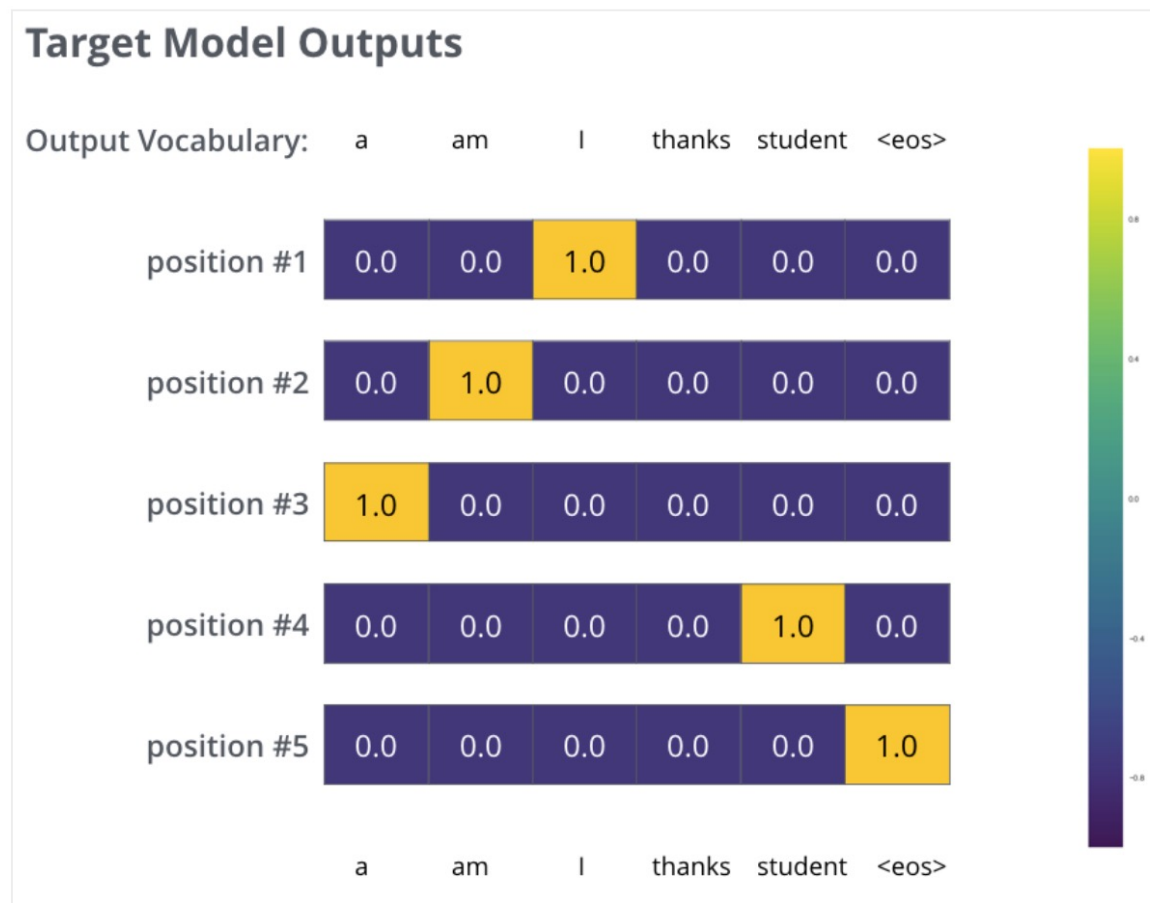
Untrained Model weights



Since the model's parameters (weights) are all initialized randomly, the (untrained) model produces a probability distribution with arbitrary values for each cell/word. We can compare it with the actual output, then tweak all the model's weights using backpropagation to make the output closer to the desired output.



Target Model weights

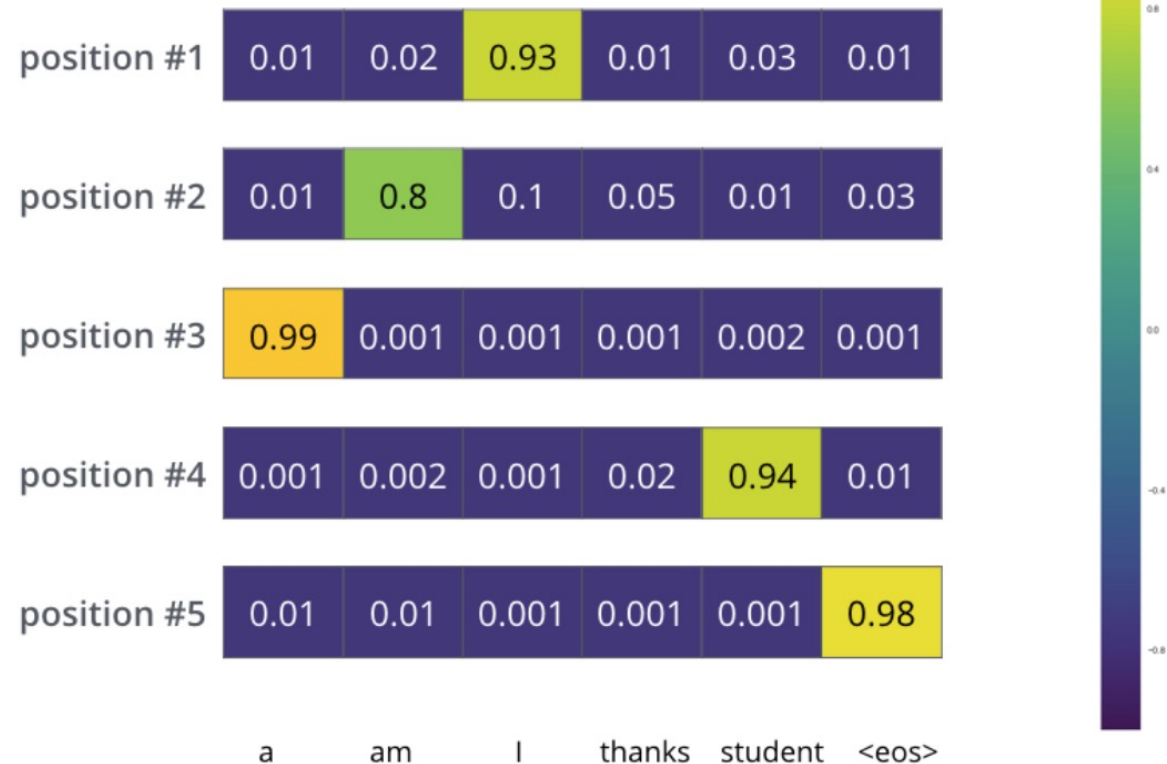


The targeted probability distributions we'll train our model against in the training example for one sample sentence.



Trained Model Outputs

Output Vocabulary: a am I thanks student <eos>



Hopefully upon training, the model would output the right translation we expect. Of course it's no real indication if this phrase was part of the training dataset (see: [cross validation](#)). Notice that every position gets a little bit of probability even if it's unlikely to be the output of that time step -- that's a very useful property of softmax which helps the training process.

